

International Cybersecurity and Digital Forensics Academy (ICDFA)

Course Code: GRC102 – Information Security Governance

Week 5 Practical Laboratory

Security Governance Practical Lab

Equifax-Style Breach Simulation: Build, Analyze, Remediate, Report

Environment: Kali Linux 2026.2 (VirtualBox), Docker/Docker Compose

Student Name: Mercy Ajayi

Registration Number: C11/26/CGRCE/17219

Date of Submission: October 11, 2026

TABLE OF CONTENTS

Executive Summary.....	1
Introduction.....	2
Scope and Authorization.....	2
Part 1: Building the Simulated Environment.....	3
Part 2: Simulating the Attack and Analyzing Governance Failures.....	22
Part 3: Implementing and Verifying Controls.....	26
Part 4: Metrics, Dashboard, and Executive Reporting.....	37

Executive Summary

This report documents a controlled simulation of the technical and governance issues behind the 2017 Equifax data breach. The aim was to recreate these conditions, identify the security failures, apply appropriate fixes, and check whether the fixes worked. The exercise was completed in four stages: setting up the vulnerable environment, simulating an attack and analyzing the governance failures, implementing and testing security controls, and preparing metrics and reports to present the results.

The initial environment contained three major weaknesses: an unpatched Apache Struts vulnerability rated Critical (CVE-2017-5638), a database using default credentials, and no network segmentation between the web and database tiers. The governance analysis identified eight distinct failures involving patch management, incident detection, policy enforcement, and other related areas.

After remediation, the vulnerability scan returned zero Critical, High, or Medium findings, compared to one in each category before the fixes. The attack, which had previously succeeded, also failed when tested again, and the number of outstanding governance failures dropped from eight to zero. Parts 1 through 4 explain the work carried out, the results obtained, and the evidence collected during the exercise.

Introduction

The 2017 Equifax data breach is an example of how multiple security weaknesses can lead to a serious data breach. Attackers exploited a known Apache Struts vulnerability that had not been patched, gaining access to the personal information of approximately 147 million people. The breach also highlighted other problems, including weak network segmentation, which could allow attackers to move between systems, and a 76-day delay before the breach was discovered.

The aim of the lab was to recreate these conditions in a small, controlled environment and observe how an attack could succeed when security controls were missing or ineffective. I then analyzed the governance failures using concepts such as policies, controls, risk ratings and metrics. After identifying the weaknesses, I implemented security fixes and tested them to confirm that they worked. This helped me understand how technical security controls and information security governance work together to reduce risk.

Scope and Authorization

All the work in this report was carried out in a private, isolated environment on my personal Kali Linux 2026.2 virtual machine in VirtualBox. I used this setup because the academy's shared Kali/Ubuntu practice hub was unavailable at the time.

The web application and database used in the lab contained only synthetic customer data, not real personal information. The vulnerable web service was accessible only through the local loopback address (127.0.0.1) and was not exposed to the internet or any network outside the lab's isolated Docker networks. I did not scan, target or access any system outside this environment.

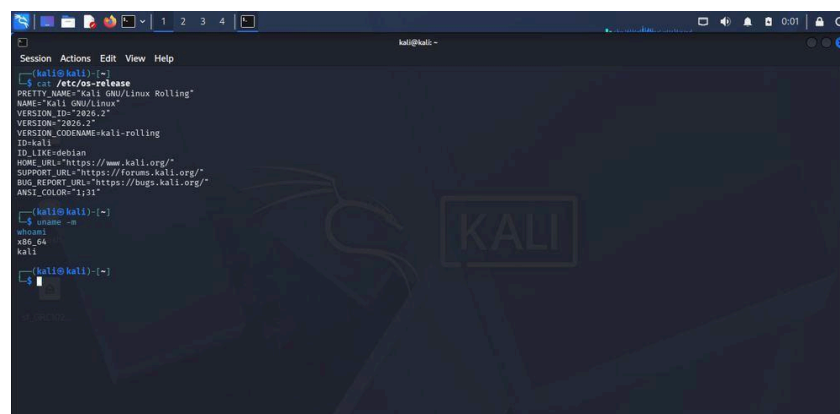
I also used Claude to help troubleshoot errors in the lab's provided scripts. Where this assistance was used, it is documented in Parts 1 through 4, in line with the course guidance on documenting AI-assisted troubleshooting.

Part 1: Building the Simulated Environment

This section explains how I set up a private and isolated Docker lab to recreate the technical conditions behind the 2017 Equifax breach. The lab contains a vulnerable Apache Struts web server (CVE-2017-5638 / S2-045), a MySQL database with synthetic customer data and default credentials, and a monitoring container. Since the academy's shared Kali/Ubuntu practice hub was unavailable, I used my personal Kali Linux 2026.2 installation in VirtualBox.

The vulnerable image tag provided in the lab file, `vulnhub/struts2:s2-045`, was not available on Docker Hub, so I used `piesecurity/apache-struts2-cve-2017-5638` instead because it targets the same vulnerability. The initial vulnerability scan confirmed the three weaknesses I intended to test: the unpatched Struts vulnerability was rated Critical, the default MySQL credentials were rated High, and the lack of network segmentation was rated Medium.

I also set up the governance tracker and added sample policies, controls, risks, incidents and metrics. While testing the dashboard, I noticed a bug in its logic. It treated every metric as if a higher value was better, which caused worsening metrics to be marked Green. I corrected this so the dashboard could display the metrics more accurately.

A screenshot of a Kali Linux terminal window. The terminal shows the output of the command `cat /etc/os-release`, which displays system information including the name 'kali GNU/Linux', version '2026.2', and architecture 'x86_64'. Below this, the command `uname -a` is executed, showing the kernel version 'x86_64' and the user 'kali'. The terminal window has a dark background with a 'KALI' logo in the center. The window title is 'kali@kali -'.

```
kali@kali:~$ cat /etc/os-release
PRETTY_NAME="kali GNU/Linux Rolling"
NAME="kali GNU/Linux"
VERSION_ID="2026.2"
VERSION="2026.2"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forum.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"

kali@kali:~$ uname -a
x86_64
kali
```

Figure 1.1: Verifying the Kali Linux version and system architecture

```

kali@kali:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:            3.8Gi      903Mi      2.4Gi          0          95Mi      2.9Gi
Swap:            0B           0B          0B           0          0B           0B
Filesystem:      79G       16G       59G      22% /
/dev/sda1:       79G       16G       59G      22% /

kali@kali:~$ ping -c 3 google.com
PING google.com (142.251.47.78) 56(84) bytes of data:
64 bytes from lclash-yb-in-f14.1e100.net (142.251.47.78): icmp_seq=1 ttl=64 time=32.5 ms
64 bytes from lclash-yb-in-f14.1e100.net (142.251.47.78): icmp_seq=2 ttl=64 time=36.4 ms
64 bytes from lclash-yb-in-f14.1e100.net (142.251.47.78): icmp_seq=3 ttl=64 time=35.6 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 32.525/34.841/36.442/1.677 ms

kali@kali:~$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [24.6 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [54.2 MB]
Fetched 76.9 MB in 3s (2.5 MB/s)
1465 packages can be upgraded. Run 'apt list --upgradable' to see them.

kali@kali:~$

```

Figure 1.2: Checking system memory, disk space, and internet connectivity

```

kali@kali:~$ sudo systemctl enable --now docker
[sudo] password for kali:
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-10-09 00:12:36 EDT; 1min 48s ago
   Invocation: e7cf3f9928ac62cd775815c7bae946
   TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
   Main PID: 671 (dockerd)
   Tasks: 9
   Memory: 185M (peak: 186.5M)
   CPU: 60ms
   CGroup: /system.slice/docker.service
           └─ /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Oct 09 00:12:35 kali dockerd[671]: time="2026-10-09T00:12:35.563780293-04:00" level=info msg="Creating a containerd client" address=/run/containerd/cont... timeout=1m0s
Oct 09 00:12:35 kali dockerd[671]: time="2026-10-09T00:12:35.570663781-04:00" level=info msg="[graphdriver] using prior storage driver: overlay2"
Oct 09 00:12:35 kali dockerd[671]: time="2026-10-09T00:12:35.581888842-04:00" level=info msg="Loading containers: start."
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.836487273-04:00" level=info msg="Loading containers: done."
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.918538867-04:00" level=info msg="Docker daemon" commit=3ef3c07c883be49a2e8897bcfd6d2952fcd7..28.5.2-dfsg4
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.928028426-04:00" level=info msg="Initializing buildkit"
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.935983742-04:00" level=info msg="Completed buildkit initialization"
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.943781882-04:00" level=info msg="Daemon has completed initialization"
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.943982714-04:00" level=info msg="API listen on /run/docker.sock"
Oct 09 00:12:36 kali systemd[1]: Started docker.service - Docker Application Container Engine.
Hint: Some lines were ellipsized, use -l to show in full.

kali@kali:~$ sudo usermod -sG docker $USER

```

Figure 1.3: Updating package repositories with apt update

```

kali@kali:~$ sudo systemctl status docker --no-pager
[sudo] password for kali:
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-10-09 00:12:36 EDT; 1min 48s ago
   Invocation: e7cf3f9928ac62cd775815c7bae946
   TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
   Main PID: 671 (dockerd)
   Tasks: 9
   Memory: 185M (peak: 186.5M)
   CPU: 60ms
   CGroup: /system.slice/docker.service
           └─ /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Oct 09 00:12:35 kali dockerd[671]: time="2026-10-09T00:12:35.563780293-04:00" level=info msg="Creating a containerd client" address=/run/containerd/cont... timeout=1m0s
Oct 09 00:12:35 kali dockerd[671]: time="2026-10-09T00:12:35.570663781-04:00" level=info msg="[graphdriver] using prior storage driver: overlay2"
Oct 09 00:12:35 kali dockerd[671]: time="2026-10-09T00:12:35.581888842-04:00" level=info msg="Loading containers: start."
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.836487273-04:00" level=info msg="Loading containers: done."
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.918538867-04:00" level=info msg="Docker daemon" commit=3ef3c07c883be49a2e8897bcfd6d2952fcd7..28.5.2-dfsg4
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.928028426-04:00" level=info msg="Initializing buildkit"
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.935983742-04:00" level=info msg="Completed buildkit initialization"
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.943781882-04:00" level=info msg="Daemon has completed initialization"
Oct 09 00:12:36 kali dockerd[671]: time="2026-10-09T00:12:36.943982714-04:00" level=info msg="API listen on /run/docker.sock"
Oct 09 00:12:36 kali systemd[1]: Started docker.service - Docker Application Container Engine.
Hint: Some lines were ellipsized, use -l to show in full.

kali@kali:~$ sudo usermod -sG docker $USER

kali@kali:~$ newgrp docker
kali@kali:~$

```

Figure 1.4: Updating package repositories with apt update


```
Session Actions Edit View Help
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

(kali@kali)~$ docker --version
docker --version
docker-compose --version
python3 -c "import requests, matplotlib; print('python libs ok!')"
Docker version 20.10.11, build 9cde435
Docker Compose version 2.0.1
python libs ok

(kali@kali)~$ mkdir -p ~/security_governance_lab
cd ~/security_governance_lab
pwd
/home/kali/security_governance_lab

(kali@kali)~/security_governance_lab$
```

Figure 1.8: Installing Docker, Docker Compose, and Python dependencies

```
Session Actions Edit View Help

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

(kali@kali)~$ docker --version
docker --version
docker-compose --version
python3 -c "import requests, matplotlib; print('python libs ok!')"
Docker version 20.10.11, build 9cde435
Docker Compose version 2.0.1
python libs ok

(kali@kali)~$ mkdir -p ~/security_governance_lab
cd ~/security_governance_lab
pwd
/home/kali/security_governance_lab

(kali@kali)~/security_governance_lab$ docker network ls
cc029b3bdc5a78a35a149b7956579ba3466f204d028dc01c9328aa9c38ab
NETWORK ID NAME DRIVER SCOPE
2f63a1a6171c bridge bridge local
6a8b5e407dca host host local
64dc335cd188 none null local
cc029b3bdc5a secgov_network bridge local

(kali@kali)~/security_governance_lab$
```

Figure 1.9: Installing Docker, Docker Compose, and Python dependencies

```
Session Actions Edit View Help

(kali@kali)~/security_governance_lab$ docker network create secgov_network
docker network ls
cc029b3bdc5a78a35a149b7956579ba3466f204d028dc01c9328aa9c38ab
NETWORK ID NAME DRIVER SCOPE
2f63a1a6171c bridge bridge local
6a8b5e407dca host host local
64dc335cd188 none null local
cc029b3bdc5a secgov_network bridge local

(kali@kali)~/security_governance_lab$ mkdir -p database_init_monitoring
cd > database_init_monitoring
cat > database_init.sql << 'EOF'
USE customer_data;

CREATE TABLE customers (
  id INT AUTO-INCREMENT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  ssn VARCHAR(11),
  dob DATE,
  address VARCHAR(100),
  city VARCHAR(50),
  state VARCHAR(2),
  zip VARCHAR(10),
  email VARCHAR(100),
  phone VARCHAR(15),
  credit_card VARCHAR(16)
);

INSERT INTO customers (first_name, last_name, dob, address, city, state, zip, email, phone, credit_card)
VALUES
('John', 'Doe', '123-45-6789', '1980-01-15', '123 Main St', 'Anytown', 'CA', '12345', 'john.doe@example.com', '555-123-4567', '4111111111111111');

EOF

(kali@kali)~/security_governance_lab$
```

Figure 1.10: Installing Docker, Docker Compose, and Python dependencies

```
kali@kali: ~/security_governance_lab
$ cat docker-compose.yml
docker-compose config --quiet && echo "compose file is valid"
services:
  web:
    image: vulhub/struts2:s2-045
    container_name: web_server
    ports:
      - "127.0.0.1:8080:8080"
    networks:
      - secgov_network
  database:
    image: mysql:5.7
    container_name: database_server
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: customer_data
      MYSQL_USER: app_user
      MYSQL_PASSWORD: app_password
    volumes:
      - ./database_init:/docker-entrypoint-initdb.d
    networks:
      - secgov_network
  monitoring:
    image: ubuntu:20.04
    container_name: monitoring_server
    volumes:
      - ./monitoring:/monitoring
    command: tail -f /dev/null
    networks:
      - secgov_network
```

Figure 1.11: Confirming package installation progress

```
kali@kali: ~/security_governance_lab
$ docker-compose up -d
[+] Running 3/3
! monitoring Interrupted 7.8s
! web error Interrupted 7.8s
! database Interrupted 7.8s
Error response from daemon: manifest for vulhub/struts2:s2-045 not found: manifest unknown: manifest unknown
kali@kali: ~/security_governance_lab
```

NETWORK ID	NAME	DRIVER	SCOPE
2f63a1a4171c	bridge	bridge	local
6a6b3e4070ca	host	host	local
64dc335cd18b	none	null	local
cc02903be8c5	secgov_network	bridge	local

Figure 1.12: Starting and enabling the Docker service

```
kali@kali: ~/security_governance_lab
$ cat docker-compose.yml
docker-compose config --quiet && echo "compose file is valid"
services:
  web:
    image: vulhub/struts2:s2-045
    container_name: web_server
    ports:
      - "127.0.0.1:8080:8080"
    networks:
      - secgov_network
  database:
    image: mysql:5.7
    container_name: database_server
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: customer_data
      MYSQL_USER: app_user
      MYSQL_PASSWORD: app_password
    volumes:
      - ./database_init:/docker-entrypoint-initdb.d
    networks:
      - secgov_network
  monitoring:
    image: ubuntu:20.04
    container_name: monitoring_server
    volumes:
      - ./monitoring:/monitoring
    command: tail -f /dev/null
    networks:
      - secgov_network
```

Figure 1.13: Rebooting after system library upgrades

```

kali@kali: ~/security_governance_lab
❯ docker-compose config --quiet && echo "compose file is valid"
compose file is valid
❯ docker-compose up -d
❯ Running 29/29
✓ web Pulled
  ✓ 9f8780b4722 Pull complete
  ✓ d3942a7a2d2 Pull complete
  ✓ 2b95970c8f9 Pull complete
  ✓ 4c75979b0c1 Pull complete
  ✓ 5f5ed4ce77dc Pull complete
  ✓ d083b20f18f Pull complete
  ✓ 1138b1f02281 Pull complete
  ✓ f027813154f1 Pull complete
  ✓ 41c16a6ed99c Pull complete
  ✓ 08038cd0803c Pull complete
  ✓ d4af2280ba7 Pull complete
  ✓ a8416d3a681b Pull complete
  ✓ 5384b5d5d8b7 Pull complete
  ✓ 4cd5bce33b9 Pull complete
✓ database Pulled
  ✓ 28e4dca84c9 Pull complete
  ✓ 1c56c3d4ce74 Pull complete
  ✓ e9f0a1c24ce Pull complete
  ✓ 68c3899c2915 Pull complete
  ✓ 6b95a940e706 Pull complete
  ✓ 9080b0b5d5de Pull complete
  ✓ ae71319c3779 Pull complete
  ✓ ffc9e9d9fd88 Pull complete
  ✓ 43d84928f8a Pull complete
  ✓ 064b2d298f8a Pull complete
  ✓ d99a4d83569b Pull complete
✓ monitoring Pulled
  ✓ 13b7e938469f Pull complete

```

Figure 1.14: Verifying the Docker service is active after reboot

```

kali@kali: ~/security_governance_lab
❯ docker-compose up -d
❯ Running 29/29
✓ web Pulled
  ✓ 9f8780b4722 Pull complete
  ✓ d3942a7a2d2 Pull complete
  ✓ 2b95970c8f9 Pull complete
  ✓ 4c75979b0c1 Pull complete
  ✓ 5f5ed4ce77dc Pull complete
  ✓ d083b20f18f Pull complete
  ✓ 1138b1f02281 Pull complete
  ✓ f027813154f1 Pull complete
  ✓ 41c16a6ed99c Pull complete
  ✓ 08038cd0803c Pull complete
  ✓ d4af2280ba7 Pull complete
  ✓ a8416d3a681b Pull complete
  ✓ 5384b5d5d8b7 Pull complete
  ✓ 4cd5bce33b9 Pull complete
✓ database Pulled
  ✓ 28e4dca84c9 Pull complete
  ✓ 1c56c3d4ce74 Pull complete
  ✓ e9f0a1c24ce Pull complete
  ✓ 68c3899c2915 Pull complete
  ✓ 6b95a940e706 Pull complete
  ✓ 9080b0b5d5de Pull complete
  ✓ ae71319c3779 Pull complete
  ✓ ffc9e9d9fd88 Pull complete
  ✓ 43d84928f8a Pull complete
  ✓ 064b2d298f8a Pull complete
  ✓ d99a4d83569b Pull complete
✓ monitoring Pulled
  ✓ 13b7e938469f Pull complete

```

Figure 1.15: Verifying the Docker service is active after reboot

```

kali@kali: ~/security_governance_lab
❯ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
294f8b2c2a05   alpine:3.8    "tail -f /dev/null"     About a minute ago Up About a minute                               monitoring_serve
b29595d18e0   mysql:5.7     "docker-entrypoint.s..." About a minute ago Up About a minute 3306/tcp, 33060/tcp     database_server
4b75a9be3833   piasecurity/apache-struts2-cve-2017-3638 "catalina.sh run"       About a minute ago Up About a minute 127.0.0.1:8080->8080/tcp web_server

```

Figure 1.16: Adding the user to the docker group and testing with hello-world

```

kali@kali:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
29658bd2ab05   ubuntu:20.04   "tail -f /dev/null"     About a minute ago   Up About a minute   3306/tcp, 33060/tcp      database_server
4b75a9be3333   piasecurity/apache-struts2-cve-2017-5638   "catalina.sh run"       About a minute ago   Up About a minute   127.0.0.1:8080->8080/tcp  web_server

kali@kali:~$ docker exec database_server mysql -uroot -ppassword -e "SELECT id, first_name, last_name, email FROM customer_data.customers;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+-----+-----+-----+
| id | first_name | last_name | email |
+----+-----+-----+-----+
| 1  | John      | Doe       | john.doe@example.com |
| 2  | Jane      | Smith     | jane.smith@example.com |
| 3  | Bob       | Johnson   | bob.johnson@example.com |
+----+-----+-----+-----+

kali@kali:~$ docker exec database_server mysql -uroot -ppassword -e "SELECT id, first_name, last_name, email FROM customer_data.customers;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+-----+-----+-----+
| id | first_name | last_name | email |
+----+-----+-----+-----+
| 1  | John      | Doe       | john.doe@example.com |
| 2  | Jane      | Smith     | jane.smith@example.com |
| 3  | Bob       | Johnson   | bob.johnson@example.com |
+----+-----+-----+-----+

```

Figure 1.17: Confirming Docker, Docker Compose, and Python library versions

```

kali@kali:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
29658bd2ab05   ubuntu:20.04   "tail -f /dev/null"     About a minute ago   Up About a minute   3306/tcp, 33060/tcp      database_server
4b75a9be3333   piasecurity/apache-struts2-cve-2017-5638   "catalina.sh run"       About a minute ago   Up About a minute   127.0.0.1:8080->8080/tcp  web_server

kali@kali:~$ docker exec database_server mysql -uroot -ppassword -e "SELECT id, first_name, last_name, email FROM customer_data.customers;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+-----+-----+-----+
| id | first_name | last_name | email |
+----+-----+-----+-----+
| 1  | John      | Doe       | john.doe@example.com |
| 2  | Jane      | Smith     | jane.smith@example.com |
| 3  | Bob       | Johnson   | bob.johnson@example.com |
+----+-----+-----+-----+

kali@kali:~$ docker exec database_server mysql -uroot -ppassword -e "SELECT id, first_name, last_name, email FROM customer_data.customers;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+-----+-----+-----+
| id | first_name | last_name | email |
+----+-----+-----+-----+
| 1  | John      | Doe       | john.doe@example.com |
| 2  | Jane      | Smith     | jane.smith@example.com |
| 3  | Bob       | Johnson   | bob.johnson@example.com |
+----+-----+-----+-----+

```

Figure 1.18: Confirming Docker, Docker Compose, and Python library versions

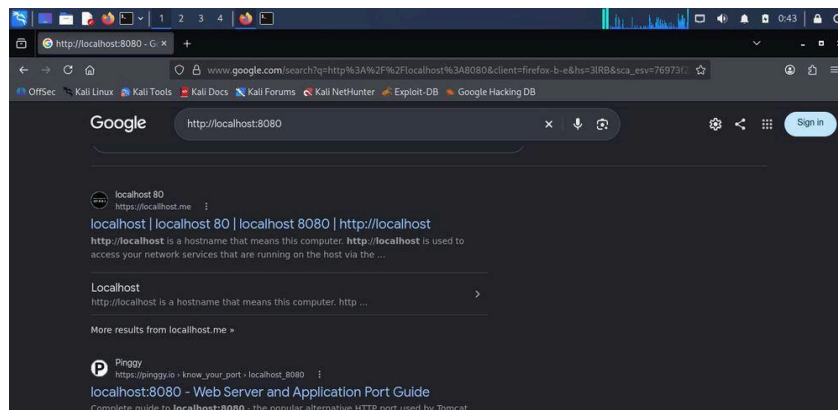


Figure 1.19: Attempting to open the Struts2 application in a browser

```
(kali@kali):~/security_governance_lab$ python3 vulnerability_scanner.py
>...
    "medium": sum(1 for r in results if r.get("severity") == "Medium"),
    "low": sum(1 for r in results if r.get("severity") == "Low"),
    "unknown": sum(1 for r in results if r.get("severity") == "Unknown")
}
}

return report

def main():
    results = []
    results.append(check_struts_vulnerability())
    results.append(check_mysql_vulnerability())
    results.append(check_network_segmentation())

    report = generate_report(results)

    report_file = f"vulnerability_report_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
    with open(report_file, "w") as f:
        json.dump(report, f, indent=2)

    print(f"Vulnerability scan completed. Report saved to {report_file}")
    print(f"Summary: {report['summary']} | Total: {report['total']} vulnerabilities found")
    print(f"Critical: {report['summary']} | {report['critical']}")
    print(f"High: {report['summary']} | {report['high']}")
    print(f"Medium: {report['summary']} | {report['medium']}")
    print(f"Low: {report['summary']} | {report['low']}")
    print(f"Unknown: {report['summary']} | {report['unknown']}")

if __name__ == "__main__":
    main()

chmod +x vulnerability_scanner.py
```

Figure 1.20: Creating the lab working directory and Docker network

```
(kali@kali):~/security_governance_lab$ python3 vulnerability_scanner.py
>...
    "medium": sum(1 for r in results if r.get("severity") == "Medium"),
    "low": sum(1 for r in results if r.get("severity") == "Low"),
    "unknown": sum(1 for r in results if r.get("severity") == "Unknown")
}
}

return report

def main():
    results = []
    results.append(check_struts_vulnerability())
    results.append(check_mysql_vulnerability())
    results.append(check_network_segmentation())

    report = generate_report(results)

    report_file = f"vulnerability_report_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
    with open(report_file, "w") as f:
        json.dump(report, f, indent=2)

    print(f"Vulnerability scan completed. Report saved to {report_file}")
    print(f"Summary: {report['summary']} | Total: {report['total']} vulnerabilities found")
    print(f"Critical: {report['summary']} | {report['critical']}")
    print(f"High: {report['summary']} | {report['high']}")
    print(f"Medium: {report['summary']} | {report['medium']}")
    print(f"Low: {report['summary']} | {report['low']}")
    print(f"Unknown: {report['summary']} | {report['unknown']}")

if __name__ == "__main__":
    main()

chmod +x vulnerability_scanner.py
```

Figure 1.21: Creating the lab working directory and Docker network

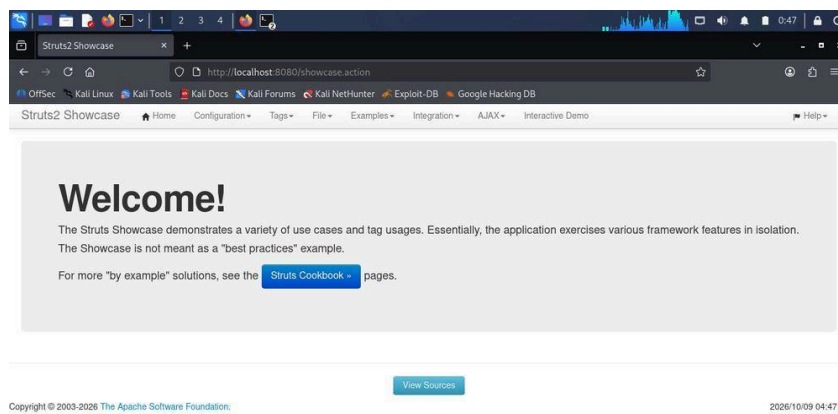
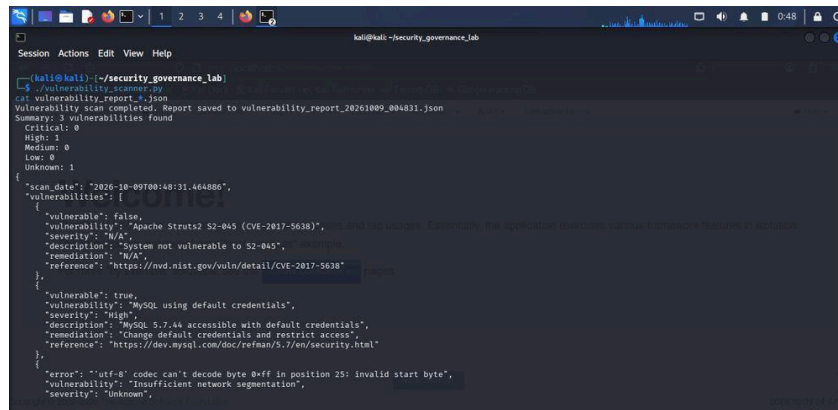
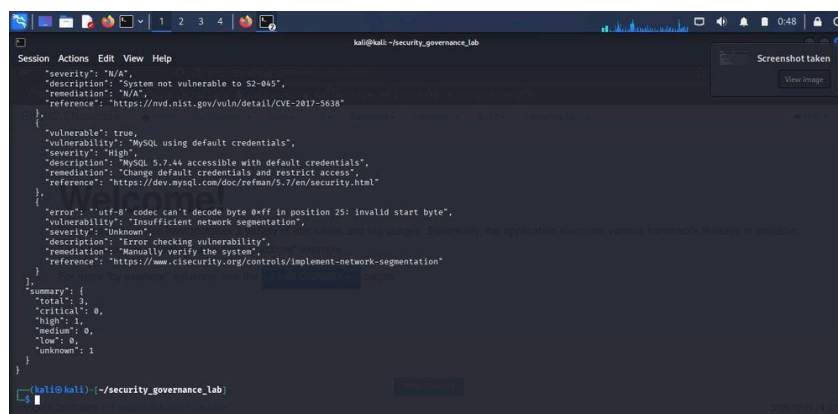


Figure 1.22: Viewing the Struts2 Showcase application in the browser



```
(kali@kali): (~/security_governance_lab)
$ ./vulnerability_scanner.py
cat vulnerability_report_*.json
Vulnerability scan completed. Report saved to vulnerability_report_20261009_004831.json
Summary: 3 vulnerabilities found
Critical: 0
High: 1
Medium: 0
Low: 0
Unknown: 1
{
  "scan_date": "2026-10-09T00:48:31.464886",
  "vulnerabilities": [
    {
      "vulnerable": false,
      "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
      "severity": "N/A",
      "description": "System not vulnerable to S2-045",
      "remediation": "N/A",
      "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
    },
    {
      "vulnerable": true,
      "vulnerability": "MySQL using default credentials",
      "severity": "High",
      "description": "MySQL 5.7.44 accessible with default credentials",
      "remediation": "Change default credentials and restrict access",
      "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
    },
    {
      "error": "utf-8 codec can't decode byte 0xff in position 25: invalid start byte",
      "vulnerability": "Insufficient network segmentation",
      "severity": "Unknown"
    }
  ],
  "summary": {
    "total": 3,
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0,
    "unknown": 1
  }
}
```

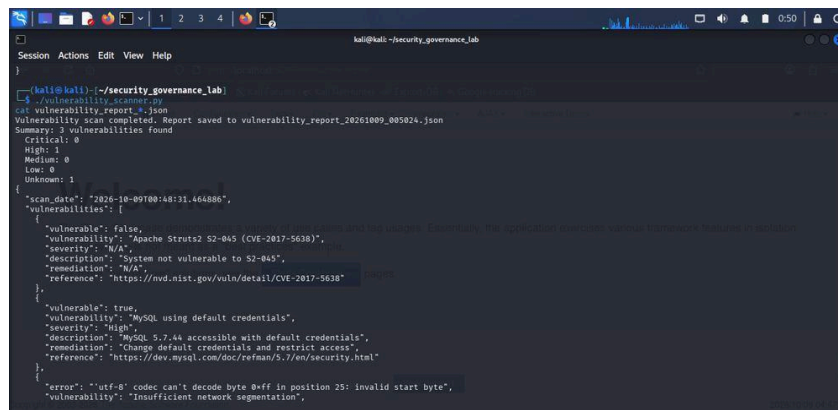
Figure 1.23: Writing the database initialization script and Docker Compose file



```
"severity": "N/A",
"description": "System not vulnerable to S2-045",
"remediation": "N/A",
"reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
},
{
  "vulnerable": true,
  "vulnerability": "MySQL using default credentials",
  "severity": "High",
  "description": "MySQL 5.7.44 accessible with default credentials",
  "remediation": "Change default credentials and restrict access",
  "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
},
{
  "error": "utf-8 codec can't decode byte 0xff in position 25: invalid start byte",
  "vulnerability": "Insufficient network segmentation",
  "severity": "Unknown",
  "description": "Error checking vulnerability",
  "remediation": "Manually verify the system",
  "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
}
],
"summary": {
  "total": 3,
  "critical": 0,
  "high": 1,
  "medium": 0,
  "low": 0,
  "unknown": 1
}
}

(kali@kali): (~/security_governance_lab)
```

Figure 1.24: Writing the database initialization script and Docker Compose file



```
(kali@kali): (~/security_governance_lab)
$ ./vulnerability_scanner.py
cat vulnerability_report_*.json
Vulnerability scan completed. Report saved to vulnerability_report_20261009_005024.json
Summary: 3 vulnerabilities found
Critical: 0
High: 1
Medium: 0
Low: 0
Unknown: 1
{
  "scan_date": "2026-10-09T00:48:31.464886",
  "vulnerabilities": [
    {
      "vulnerable": false,
      "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
      "severity": "N/A",
      "description": "System not vulnerable to S2-045",
      "remediation": "N/A",
      "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
    },
    {
      "vulnerable": true,
      "vulnerability": "MySQL using default credentials",
      "severity": "High",
      "description": "MySQL 5.7.44 accessible with default credentials",
      "remediation": "Change default credentials and restrict access",
      "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
    },
    {
      "error": "utf-8 codec can't decode byte 0xff in position 25: invalid start byte",
      "vulnerability": "Insufficient network segmentation",
      "severity": "Unknown"
    }
  ],
  "summary": {
    "total": 3,
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0,
    "unknown": 1
  }
}
```

Figure 1.25: Validating the Docker Compose configuration

```

{
  "remediation": "Manually verify the system",
  "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
},
{
  "summary": {
    "total": 3,
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0,
    "unknown": 1
  }
},
{
  "scan_date": "2020-10-09T00:50:24.117998",
  "vulnerabilities": [
    {
      "vulnerable": false,
      "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
      "severity": "N/A",
      "description": "System not vulnerable to S2-045",
      "remediation": "N/A",
      "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
    },
    {
      "vulnerable": true,
      "vulnerability": "MySQL using default credentials",
      "severity": "High",
      "description": "MySQL 5.7.44 accessible with default credentials",
      "remediation": "Change default credentials and restrict access",
      "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
    },
    {
      "error": "utf-8 codec can't decode byte 0xff in position 25: invalid start byte",
      "vulnerability": "Insufficient network segmentation",
      "severity": "Unknown",
      "description": "Error checking vulnerability",
      "remediation": "Manually verify the system",
      "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
    }
  ]
}

```

Figure 1.26: Starting the three lab containers (web, database, monitoring)

```

{
  "vulnerable": false,
  "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
  "severity": "N/A",
  "description": "System not vulnerable to S2-045",
  "remediation": "N/A",
  "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
},
{
  "vulnerable": true,
  "vulnerability": "MySQL using default credentials",
  "severity": "High",
  "description": "MySQL 5.7.44 accessible with default credentials",
  "remediation": "Change default credentials and restrict access",
  "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
},
{
  "error": "utf-8 codec can't decode byte 0xff in position 25: invalid start byte",
  "vulnerability": "Insufficient network segmentation",
  "severity": "Unknown",
  "description": "Error checking vulnerability",
  "remediation": "Manually verify the system",
  "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
},
{
  "summary": {
    "total": 3,
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0,
    "unknown": 1
  }
}

```

Figure 1.27: Starting the three lab containers (web, database, monitoring)

```

(kali@kali:~/security_governance_lab)
$ grep -n target_url vulnerability_scanner.py
19: target_url = "http://localhost:8080"
20: response = requests.get(target_url, headers=headers, timeout=5)
21: if response.status_code == 200:
22:     print(True)
23: else:
24:     print(False)
25: if result.stdout == 0:
26:     print(result.stdout or "") + (result.stderr or "")
27:
(kali@kali:~/security_governance_lab)

```

Figure 1.28: Confirming container images pulling and starting

```

Session Actions Edit View Help
kali@kali: ~/security_governance_lab

{
  "summary": {
    "total": 3,
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0,
    "unknown": 1
  }
}

(kali@kali)~/security_governance_lab
$ grep -n "target_url" vulnerability_scanner.py
grep -n "text" vulnerability_scanner.py
grep -n "combined" vulnerability_scanner.py
grep -n "if result.returncode" vulnerability_scanner.py
12: target_url = "http://localhost:8080"
19: response = requests.get(target_url, headers=headers, timeout=5)
55: test=True
94: test=True
97: combined = (result.stdout or "") + (result.stderr or "")
58: if result.returncode == 0:

(kali@kali)~/security_governance_lab
$ sed -i '12s/http://localhost:8080/http://localhost:8080/showcase.action/' vulnerability_scanner.py
sed -i '94s/text=True/text=False/' vulnerability_scanner.py
sed -i '97s/.* combined = (result.stdout or "") + (result.stderr or b"") vulnerability_scanner.py

(kali@kali)~/security_governance_lab
$ grep -n "bad header" vulnerability_scanner.py
99: if "bad header" in combined.lower() or result.returncode in (0, 8) or combined.strip() != "":

(kali@kali)~/security_governance_lab

```

Figure 1.29: Checking running containers with docker ps

```

Session Actions Edit View Help
kali@kali: ~/security_governance_lab

19: response = requests.get(target_url, headers=headers, timeout=5)
55: test=True
94: test=True
97: combined = (result.stdout or "") + (result.stderr or "")
58: if result.returncode == 0:

(kali@kali)~/security_governance_lab
$ sed -i '12s/http://localhost:8080/http://localhost:8080/showcase.action/' vulnerability_scanner.py
sed -i '94s/text=True/text=False/' vulnerability_scanner.py
sed -i '97s/.* combined = (result.stdout or "") + (result.stderr or b"") vulnerability_scanner.py

(kali@kali)~/security_governance_lab
$ grep -n "bad header" vulnerability_scanner.py
99: if "bad header" in combined.lower() or result.returncode in (0, 8) or combined.strip() != "":

(kali@kali)~/security_governance_lab
$ sed -i '99s/.* if result.returncode in (0, 8) or len(combined.strip()) > 0:/' vulnerability_scanner.py

(kali@kali)~/security_governance_lab
$ grep -n "target_url" vulnerability_scanner.py
12: target_url = "http://localhost:8080/showcase.action"
19: response = requests.get(target_url, headers=headers, timeout=5)
55: test=True
94: test=True
97: combined = (result.stdout or "") + (result.stderr or b"")
58: if result.returncode == 0:

(kali@kali)~/security_governance_lab

```

Figure 1.30: Querying the database to confirm sample customer records loaded

```

Session Actions Edit View Help
kali@kali: ~/security_governance_lab

return {

(kali@kali)~/security_governance_lab
$ rm vulnerability_report_*.json
./vulnerability_scanner.py
out vulnerability_report_*.json
Vulnerability scan completed. Report saved to vulnerability_report_20261009_085552.json
Summary: 3 vulnerabilities found
Critical: 1
High: 1
Medium: 1
Low: 0
Unknown: 0
{
  "scan_date": "2026-10-09T08:55:52.058994",
  "vulnerabilities": [
    {
      "vulnerable": true,
      "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
      "severity": "Critical",
      "description": "Remote Code Execution vulnerability in Apache Struts2",
      "remediation": "Upgrade to Struts 2.3.32 or Struts 2.5.10.1",
      "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
    },
    {
      "vulnerable": true,
      "vulnerability": "MySQL using default credentials",
      "severity": "High",
      "description": "MySQL 5.7.44 accessible with default credentials",
      "remediation": "Change default credentials and restrict access",
      "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
    },
    {
      "vulnerable": true,

```

Figure 1.31: Writing and running the baseline vulnerability scanner

```

Session Actions Edit View Help
kali@kali: ~security_governance_lab

"vulnerabilities": [
  {
    "vulnerable": true,
    "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
    "severity": "Critical",
    "description": "Remote Code Execution vulnerability in Apache Struts2",
    "remediation": "Update to Struts 2.3.32 or Struts 2.5.10.1",
    "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
  },
  {
    "vulnerable": true,
    "vulnerability": "MySQL using default credentials",
    "severity": "High",
    "description": "MySQL 5.7.44 accessible with default credentials",
    "remediation": "Change default credentials and restrict access",
    "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
  },
  {
    "vulnerable": true,
    "vulnerability": "Insufficient network segmentation",
    "severity": "Medium",
    "description": "Web server can directly reach database server on the shared network",
    "remediation": "Implement network segmentation and access controls",
    "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
  }
],
"summary": {
  "total": 3,
  "critical": 1,
  "high": 1,
  "medium": 1,
  "low": 0,
  "unknown": 0
}

```

Figure 1.32: Writing and running the baseline vulnerability scanner

```

Session Actions Edit View Help
kali@kali: ~security_governance_lab

if result.returncode in (0, 0) or len(combined.strip()) > 0:
    return 1

(kali@kali) ~/security_governance_lab
$ ./vulnerability_report.py
cat vulnerability_report.json
Vulnerability scan completed. Report saved to vulnerability_report_20261009_005655.json
Summary: 3 vulnerabilities found
Critical: 1
High: 1
Medium: 1
Low: 0
Unknown: 0

"scan_date": "2026-10-09T00:56:55.075177", a variety of low-level and high-level changes. Essentially, the application executes various framework features in parallel
"vulnerabilities": [
  {
    "vulnerable": true,
    "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
    "severity": "Critical",
    "description": "Remote Code Execution vulnerability in Apache Struts2",
    "remediation": "Update to Struts 2.3.32 or Struts 2.5.10.1",
    "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
  },
  {
    "vulnerable": true,
    "vulnerability": "MySQL using default credentials",
    "severity": "High",
    "description": "MySQL 5.7.44 accessible with default credentials",
    "remediation": "Change default credentials and restrict access",
    "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
  },
  {
    "vulnerable": true,
    "vulnerability": "Insufficient network segmentation",
    "severity": "Medium",
    "description": "Web server can directly reach database server on the shared network",
    "remediation": "Implement network segmentation and access controls",
    "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
  }
],
"summary": {
  "total": 3,
  "critical": 1,
  "high": 1,
  "medium": 1,
  "low": 0,
  "unknown": 0
}

```

Figure 1.33: Reviewing an early baseline scan result before fixing the scanner's probe URL

```

Session Actions Edit View Help
kali@kali: ~security_governance_lab

"vulnerabilities": [
  {
    "vulnerable": true,
    "vulnerability": "MySQL using default credentials",
    "severity": "High",
    "description": "MySQL 5.7.44 accessible with default credentials",
    "remediation": "Change default credentials and restrict access",
    "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
  },
  {
    "vulnerable": true,
    "vulnerability": "Insufficient network segmentation",
    "severity": "Medium",
    "description": "Web server can directly reach database server on the shared network",
    "remediation": "Implement network segmentation and access controls",
    "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
  }
],
"summary": {
  "total": 3,
  "critical": 1,
  "high": 1,
  "medium": 1,
  "low": 0,
  "unknown": 0
}

(kali@kali) ~/security_governance_lab
$ echo "Baseline scan: vulnerability_report_20261009_005655.json" >> README_notes.txt

(kali@kali) ~/security_governance_lab
$ cat README_notes.txt
Baseline scan: vulnerability_report_20261009_005655.json

```

Figure 1.34: Reviewing an early baseline scan result before fixing the scanner's probe URL

```

Session Actions Edit View Help
parser.add_argument("--list-policies", action="store_true")
parser.add_argument("--list-controls", action="store_true")
parser.add_argument("--list-metrics", action="store_true")
parser.add_argument("--list-audits", action="store_true")
parser.add_argument("--dashboard", action="store_true")

args = parser.parse_args()
tracker = GovernanceTracker()

if args.list_policies:
    print(json.dumps(tracker.list_policies(), indent=2))
elif args.list_controls:
    print(json.dumps(tracker.list_controls(), indent=2))
elif args.list_risks:
    print(json.dumps(tracker.list_risks(), indent=2))
elif args.list_incidents:
    print(json.dumps(tracker.list_incidents(), indent=2))
elif args.list_metrics:
    print(json.dumps(tracker.list_metrics(), indent=2))
elif args.list_audits:
    print(json.dumps(tracker.list_audits(), indent=2))
elif args.dashboard:
    print(json.dumps(tracker.generate_dashboard(), indent=2))
else:
    parser.print_help()

if __name__ == "__main__":
    main()
EOF

chmod +x governance_tracker.py

kali@kali: ~/security_governance_lab

```

Figure 1.35: Reviewing an early baseline scan result before fixing the scanner's probe URL

```

Session Actions Edit View Help
if args.list_policies:
    print(json.dumps(tracker.list_policies(), indent=2))
elif args.list_controls:
    print(json.dumps(tracker.list_controls(), indent=2))
elif args.list_risks:
    print(json.dumps(tracker.list_risks(), indent=2))
elif args.list_incidents:
    print(json.dumps(tracker.list_incidents(), indent=2))
elif args.list_metrics:
    print(json.dumps(tracker.list_metrics(), indent=2))
elif args.list_audits:
    print(json.dumps(tracker.list_audits(), indent=2))
elif args.segmentation:
    print(json.dumps(tracker.segmentation(), indent=2))
else:
    parser.print_help()

if __name__ == "__main__":
    main()
EOF

chmod +x governance_tracker.py

kali@kali: ~/security_governance_lab

kali@kali: ~/security_governance_lab
$ wc -l governance_tracker.py
python3 -c 'import ast; ast.parse(open("governance_tracker.py").read()); print("syntax ok")'
222 governance_tracker.py
syntax ok

kali@kali: ~/security_governance_lab

```

Figure 1.36: Fixing the vulnerability scanner's target URL and segmentation check logic

```

Session Actions Edit View Help
syntax ok

kali@kali: ~/security_governance_lab
$ >...
Average time to remediate critical vulnerabilities (days):
7.8
12.9
"May 2023"
"Declining"
)

tracker.add_metric(
    "Security Incidents",
    "Number of security incidents in the period",
    0.9,
    1.8,
    "May 2023",
    "Stable"
)

tracker.add_audit(
    "Annual Security Audit",
    "Comprehensive audit of security controls",
    "2023-05-01",
    "External Auditor",
    "Several findings related to patch management and network segmentation",
    "Improve patch management process and implement network segmentation"
)

print("Governance tracker initialized with sample data")
print(f"Policy IDs: {patch_policy_id}, {vuln_policy_id}, {incident_policy_id}")

if __name__ == "__main__":
    initialize_governance_data()

```

Figure 1.37: Writing the governance tracker script

```
kali@kali: ~/security_governance_lab

Session Actions Edit View Help

17.0.
"May 2023".
"Declining":
)

tracker.add_metric(
    "Security Incidents",
    "Number of Security Incidents in the period",
    0.0,
    1.0,
    "May 2023",
    "Stable"
)

! Come!

tracker.add_audit(
    "Annual Security Audit",
    "Comprehensive audit of security controls",
    "2023-01-01",
    "External Auditor",
    "Several findings related to patch management and network segmentation",
    "Improve patch management process and implement network segmentation"
)

print("Governance tracker initialized with sample data")
print(f"Policy IDs: {patch_policy_id}, {vuln_policy_id}, {incident_policy_id}")

if __name__ == "__main__":
    initialize_governance_data()

EOF

chmod +x initialize_governance.py

kali@kali: ~/security_governance_lab
```

Figure 1.38: *Writing the governance tracker script*

```

kali@kali: ~/security_governance_lab
python3 -c "import ast; ast.parse(open('initialize_governance.py').read()); print('syntax ok!')"
~/initialize_governance.py
~/governance_tracker.py --dashboard
syntax ok
governance tracker initialized with sample data
Policy IDs: POL-001, POL-002, POL-003
{
  "generated_date": "2026-10-09T01:01:14.949987",
  "summary": {
    "policies": 2,
    "controls": 3,
    "risks": 3,
    "incidents": 1,
    "metrics": 3,
    "audits": 1
  },
  "risk_summary": {
    "critical": 1,
    "high": 1,
    "medium": 1,
    "low": 0,
    "very_low": 0
  },
  "incident_summary": {
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0
  },
  "metrics_summary": [
    {
      "metric": "Policy Compliance",
      "value": 85,
      "unit": "Percentage"
    },
    {
      "metric": "Incident Response Time",
      "value": 45,
      "unit": "Minutes"
    },
    {
      "metric": "Risk Score",
      "value": 15,
      "unit": "Score"
    }
  ]
}

```

Figure 1.39: *Writing the governance tracker script*

```

kali@kali:~$ security_governance_lab

Session Actions Edit View Help

{
  "policies": 3,
  "controls": 3,
  "risks": 3,
  "incidents": 1,
  "metrics": 3,
  "audits": 1
},
{
  "risk_summary": {
    "critical": 1,
    "high": 1,
    "medium": 1,
    "low": 0,
    "very_low": 0
  },
  "incident_summary": {
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0
  },
  "metrics_summary": {
    "name": "Patch Compliance",
    "target": 95.0,
    "actual": 85.0,
    "status": "Red"
  },
  {
    "name": "Vulnerability Remediation Time",
    "target": 7.0,
    "actual": 12.0,
    "status": "Green"
  }
},
{
  "name": "Incident Response Time",
  "target": 4.0,
  "actual": 3.0,
  "status": "Green"
}
}

Welcome!

This application demonstrates a variety of new controls and log sources. Externally, the application overcomes various framework features in isolation.

The metrics section shows a recent set of "best practice" examples.

The metrics section uses the "best practice" examples.

[Security Governance Lab]
[Security Governance Lab]

```

Figure 1.40: *Writing the governance tracker script*

```

kali@kali:~/security_governance_lab$ cat governance_tracker.py
#!/usr/bin/env python3
import json
import os
import sys
import time

class GovernanceTracker:
    def __init__(self):
        self.data = {}
        self.status = "Green"

    def add_metric(self, name, description, target, actual, period, trend):
        metric = {
            "name": name,
            "description": description,
            "target": target,
            "actual": actual,
            "period": period,
            "trend": trend,
            "status": "Active"
        }
        self.data["metrics"].append(metric)
        self.save_data()

    def metrics_summary(self):
        summary = {}
        for metric in self.data["metrics"]:
            status = "Green" if metric["actual"] <= metric["target"] else "Red"
            summary[metric["name"]] = status

        return summary

    def save_data(self):
        filename = "governance_data.json"
        with open(filename, "w") as f:
            json.dump(self.data, f)

    def load_data(self):
        filename = "governance_data.json"
        if os.path.exists(filename):
            with open(filename, "r") as f:
                self.data = json.load(f)
            self.status = "Green"

    def run(self):
        self.load_data()
        while True:
            self.add_metric("Patch Compliance", "Percentage of systems with patches up to date", 95.0, 85.0, 10, "Up")
            self.add_metric("Vulnerability Remediation Time", "Average time to remediate vulnerabilities", 7.0, 12.0, 10, "Down")
            self.add_metric("Security Incidents", "Number of security incidents", 1.0, 1.0, 10, "Stable")
            time.sleep(10)

gtracker = GovernanceTracker()
gtracker.run()

```

Figure 1.41: Writing the governance data initialization script

```

kali@kali:~/security_governance_lab$ cat governance_tracker.py
#!/usr/bin/env python3
import json
import os
import sys
import time

class GovernanceTracker:
    def __init__(self):
        self.data = {}
        self.status = "Green"

    def add_metric(self, name, description, target, actual, period, trend):
        metric = {
            "name": name,
            "description": description,
            "target": target,
            "actual": actual,
            "period": period,
            "trend": trend,
            "status": "Active"
        }
        self.data["metrics"].append(metric)
        self.save_data()

    def metrics_summary(self):
        summary = {}
        for metric in self.data["metrics"]:
            status = "Green" if metric["actual"] <= metric["target"] else "Red"
            summary[metric["name"]] = status

        return summary

    def save_data(self):
        filename = "governance_data.json"
        with open(filename, "w") as f:
            json.dump(self.data, f)

    def load_data(self):
        filename = "governance_data.json"
        if os.path.exists(filename):
            with open(filename, "r") as f:
                self.data = json.load(f)
            self.status = "Green"

    def run(self):
        self.load_data()
        while True:
            self.add_metric("Patch Compliance", "Percentage of systems with patches up to date", 95.0, 85.0, 10, "Up")
            self.add_metric("Vulnerability Remediation Time", "Average time to remediate vulnerabilities", 7.0, 12.0, 10, "Down")
            self.add_metric("Security Incidents", "Number of security incidents", 1.0, 1.0, 10, "Stable")
            time.sleep(10)

gtracker = GovernanceTracker()
gtracker.run()

```

Figure 1.42: Writing the governance data initialization script

```

kali@kali:~/security_governance_lab$ cat governance_tracker.py
#!/usr/bin/env python3
import json
import os
import sys
import time

class GovernanceTracker:
    def __init__(self):
        self.data = {}
        self.status = "Green"

    def add_metric(self, name, description, target, actual, period, trend):
        metric = {
            "name": name,
            "description": description,
            "target": target,
            "actual": actual,
            "period": period,
            "trend": trend,
            "status": "Active"
        }
        self.data["metrics"].append(metric)
        self.save_data()

    def metrics_summary(self):
        summary = {}
        for metric in self.data["metrics"]:
            status = "Green" if metric["actual"] <= metric["target"] else "Red"
            summary[metric["name"]] = status

        return summary

    def save_data(self):
        filename = "governance_data.json"
        with open(filename, "w") as f:
            json.dump(self.data, f)

    def load_data(self):
        filename = "governance_data.json"
        if os.path.exists(filename):
            with open(filename, "r") as f:
                self.data = json.load(f)
            self.status = "Green"

    def run(self):
        self.load_data()
        while True:
            self.add_metric("Patch Compliance", "Percentage of systems with patches up to date", 95.0, 85.0, 10, "Up")
            self.add_metric("Vulnerability Remediation Time", "Average time to remediate vulnerabilities", 7.0, 12.0, 10, "Down")
            self.add_metric("Security Incidents", "Number of security incidents", 1.0, 1.0, 10, "Stable")
            time.sleep(10)

gtracker = GovernanceTracker()
gtracker.run()

```

Figure 1.43: Running the governance tracker initialization

```
Session Actions Edit View Help
"status": "Active"
}
self.data["metrics"].append(metric)
self.save_data()

(kali@kali:~/security_governance_lab)
$ sed -i '93s/.*/\n    def add_metric(self, name, description, target, actual, period, trend, lower_is_better=False):/' governance_tracker.py
(kali@kali:~/security_governance_lab)
$ sed -i '101a\n        "lower_is_better": lower_is_better,' governance_tracker.py
(kali@kali:~/security_governance_lab)
$ sed -i '93,103p\n    governance_tracker.py\npython3 -c "import ast; ast.parse(open(governance_tracker.py).read()); print('syntax ok')"\n    def add_metric(self, name, description, target, actual, period, trend, lower_is_better=False):\n        metric = {\n            "id": f"MET-{len(self.data['metrics']) + 1:03d}",\n            "name": name,\n            "description": description,\n            "target": target,\n            "actual": actual,\n            "period": period,\n            "trend": trend,\n            "lower_is_better": lower_is_better,\n            "status": "Active"\n        }\n    '\n\nsyntax ok
(kali@kali:~/security_governance_lab)
```

Figure 1.44: Reviewing the governance dashboard output before the metrics-direction bug fix

```
Session Actions Edit View Help
self.save_data()

(kali@kali:~/security_governance_lab)
$ sed -i '93s/.*/\n    def add_metric(self, name, description, target, actual, period, trend, lower_is_better=False):/' governance_tracker.py
(kali@kali:~/security_governance_lab)
$ sed -i '101a\n        "lower_is_better": lower_is_better,' governance_tracker.py
(kali@kali:~/security_governance_lab)
$ sed -i '93,103p\n    governance_tracker.py\npython3 -c "import ast; ast.parse(open(governance_tracker.py).read()); print('syntax ok')"\n    def add_metric(self, name, description, target, actual, period, trend, lower_is_better=False):\n        metric = {\n            "id": f"MET-{len(self.data['metrics']) + 1:03d}",\n            "name": name,\n            "description": description,\n            "target": target,\n            "actual": actual,\n            "period": period,\n            "trend": trend,\n            "lower_is_better": lower_is_better,\n            "status": "Active"\n        }\n    '\n\nsyntax ok
(kali@kali:~/security_governance_lab)
$ grep -n status=Green=Red governance_tracker.py
179:         "status": "Green" if m["actual"] <= m["target"] else "Red"
(kali@kali:~/security_governance_lab)
$ sed -i '179s=.*/\n        "status": "Green" if m["actual"] <= m["target"] if m.get("lower_is_better") else m["actual"] >= m["target"] else "Red":' governance_tracker.py
(kali@kali:~/security_governance_lab)
```

Figure 1.45: Investigating the governance dashboard's incorrect metric status logic

```
Session Actions Edit View Help
"status": "Active"
}
self.data["metrics"].append(metric)
self.save_data()

(kali@kali:~/security_governance_lab)
$ grep -n status=Green=Red governance_tracker.py
179:         "status": "Green" if m["actual"] <= m["target"] else "Red"
(kali@kali:~/security_governance_lab)
$ sed -i '179s=.*/\n        "status": "Green" if m["actual"] <= m["target"] if m.get("lower_is_better") else m["actual"] >= m["target"] else "Red":' governance_tracker.py
(kali@kali:~/security_governance_lab)
$ sed -i '112,113p\n    governance_tracker.py\npython3 -c "import ast; ast.parse(open(governance_tracker.py).read()); print('syntax ok')"\n    }\n    "metrics_summary": [\n        {\n            "name": m["name"],\n            "target": m["target"],\n            "actual": m["actual"],\n            "status": "Green" if m["actual"] <= m["target"] if m.get("lower_is_better") else m["actual"] >= m["target"] else "Red"\n        }\n    for m in self.data["metrics"]\n\nsyntax ok
(kali@kali:~/security_governance_lab)
```

Figure 1.46: Locating the exact lines to fix in governance_tracker.py

[illegible]

Figure 1.47: *Locating the exact lines to fix in governance_tracker.py*

[illegible]

Figure 1.48: *Locating the exact lines to fix in governance_tracker.py*

```
kali@kali:~$ cat /etc/passwd | grep kali | cut -d ':' -f 1
```

```
kali@kali:~$ sudo su
```

```
kali@kali:~$ cd ~/security_governance_lab
```

```
kali@kali:~/security_governance_lab$ rm -f governance_data.json
```

```
kali@kali:~/security_governance_lab$ grep -n "Vulnerability Remediation Time|Security Incidents" initialize_governance.py
```

```
104:         "Vulnerability Remediation Time",
113:         "Security Incidents".
```

```
kali@kali:~/security_governance_lab$ sed -n '108,120' initialize_governance.py
```

```
Improving"
```

```
tracker.add_metric(
    "Vulnerability Remediation Time",
    "Average time to remediate critical vulnerabilities (days)",
    7.0,
    12.0,
    "May 2021",
    "Declining"
```

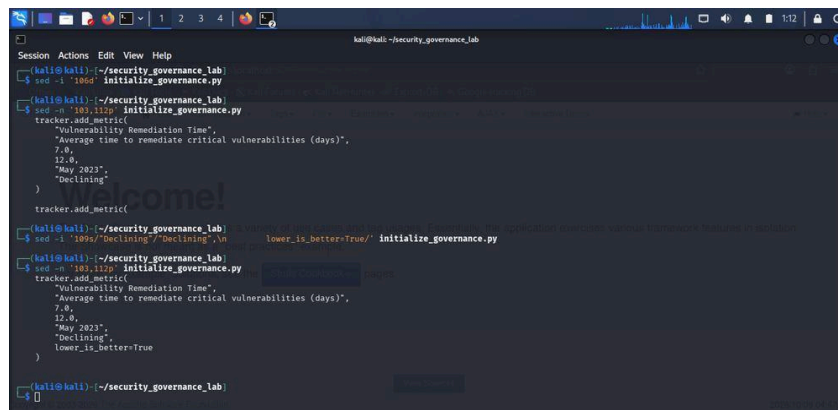
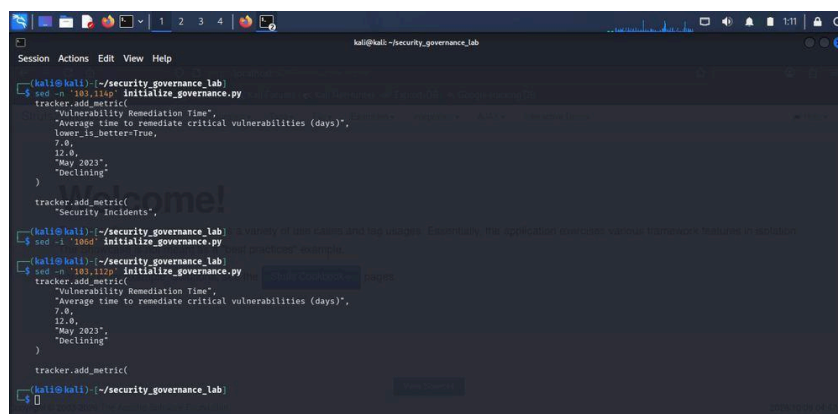
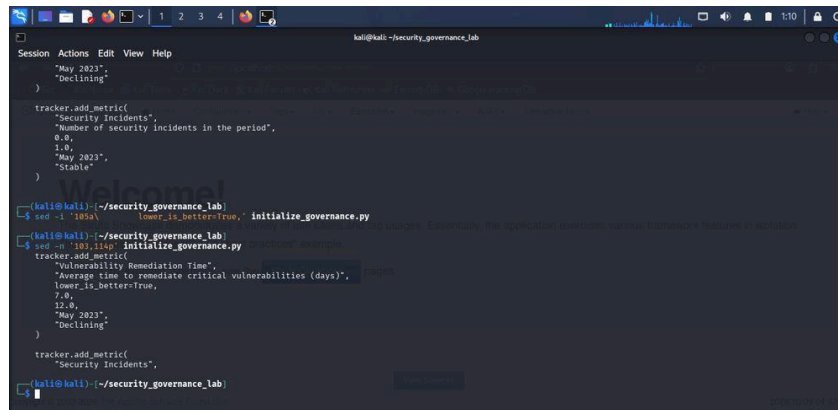
```
)
```

```
tracker.add_metric(
    "Security Incidents",
    "Number of security incidents in the period",
    0.0,
    1.0,
    "May 2021",
    "Stable"
```

```
)
```

```
kali@kali:~/security_governance_lab$
```

Figure 1.49: Applying the fix to the `governance_tracker.py` metric status logic



```

kali@kali:~/security_governance_lab
$ grep -n "Stable" initialize_governance.py
119:         "Stable",
$ sed -i '119s/"Stable"/"Stable",\n        lower_is_better=True/' initialize_governance.py
$ python3 -c "import ast; ast.parse(open('initialize_governance.py').read()); print('syntax ok!'"
syntax ok
kali@kali:~/security_governance_lab
$

```

Figure 1.53: Re-running governance initialization and confirming the corrected dashboard

```

kali@kali:~/security_governance_lab
$ python3 governance_tracker.py --dashboard
Governance tracker initialized with sample data
Policy IDs: POL-001, POL-002, POL-003
{
  "generated_date": "2026-10-09T01:13:55.302328",
  "summary": {
    "policies": 3,
    "controls": 3,
    "risks": 3,
    "incidents": 1,
    "metrics": 3,
    "audits": 1
  },
  "risk_summary": {
    "critical": 1,
    "high": 1,
    "medium": 1,
    "low": 0,
    "very_low": 0
  },
  "incident_summary": {
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0
  },
  "metrics_summary": [
    {
      "name": "Patch Compliance",
      "target": 95.0,
      "actual": 85.0,
      "status": "Red"
    },
    {
      "name": "Vulnerability Remediation Time",
      "target": 7.0,
      "actual": 12.0,
      "status": "Red"
    },
    {
      "name": "Security Incidents",
      "target": 0.0,
      "actual": 1.0,
      "status": "Red"
    }
  ]
}

```

Figure 1.54: Re-running governance initialization and confirming the corrected dashboard

```

kali@kali:~/security_governance_lab
$ python3 governance_tracker.py --dashboard
Governance tracker initialized with sample data
Policy IDs: POL-001, POL-002, POL-003
{
  "generated_date": "2026-10-09T01:13:55.302328",
  "summary": {
    "policies": 3,
    "controls": 3,
    "risks": 3,
    "incidents": 1,
    "metrics": 3,
    "audits": 1
  },
  "risk_summary": {
    "critical": 1,
    "high": 1,
    "medium": 1,
    "low": 0,
    "very_low": 0
  },
  "incident_summary": {
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0
  },
  "metrics_summary": [
    {
      "name": "Patch Compliance",
      "target": 95.0,
      "actual": 85.0,
      "status": "Red"
    },
    {
      "name": "Vulnerability Remediation Time",
      "target": 7.0,
      "actual": 12.0,
      "status": "Red"
    },
    {
      "name": "Security Incidents",
      "target": 0.0,
      "actual": 1.0,
      "status": "Red"
    }
  ]
}

```

Figure 1.55: Re-running governance initialization and confirming the corrected dashboard

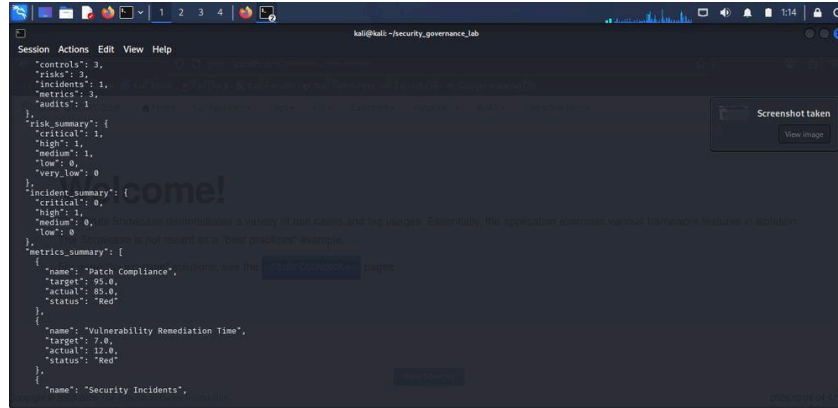
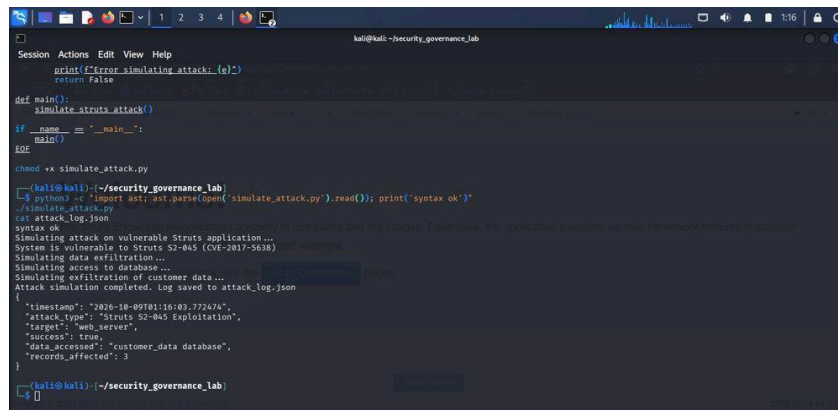


Figure 1.56: Re-running governance initialization and confirming the corrected dashboard

Part 2: Simulating the Attack and Analyzing Governance Failures

I carried out a simulated attack against the unpatched environment and successfully exploited the Struts vulnerability to access the customer database. The attack went undetected, reflecting the 76-day detection gap in the actual Equifax breach. I then used a governance failure analysis script to compare the vulnerability scan results, attack log and governance register. This identified eight distinct failures relating to patch management, incident detection, policy implementation, vulnerability management, metrics and measurement, security architecture, and authentication and access control.

From this stage, I learned that security failures can build on one another. The attack did not succeed because of just one missing control. Patches had not been applied, default credentials were still in use, network segmentation was absent, and monitoring failed to detect the attack. I also learned that having a policy recorded in the governance register does not mean it is being followed. A policy only helps protect an organisation when it is properly implemented and enforced.



```
Session Actions Edit View Help
kali@kali: ~/security_governance_lab
print(f'Error simulating attack: {e}')
return False

def main():
    simulate_struts_attack()
    return True

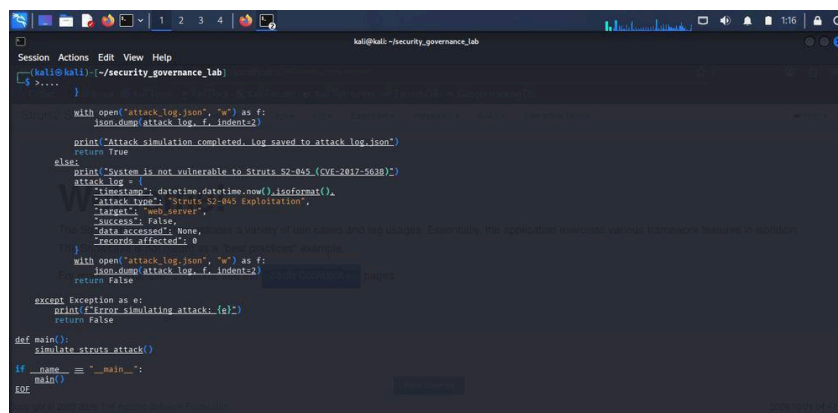
if __name__ == '__main__':
    main()
EOF

chmod +x simulate_attack.py

kali@kali:~/security_governance_lab
$ python3 -c 'import ast; ast.parse(open('simulate_attack.py').read()); print('syntax ok')'
/simulate_attack.py
cat attack_log.json
syntax ok
Simulating attack on vulnerable Struts application...
System is vulnerable to Struts S2-045 (CVE-2017-5638)
Simulating data exfiltration...
Simulating access to database...
Simulating exfiltration of customer data...
Attack simulation completed. Log saved to attack_log.json
{
  "timestamp": "2024-10-08T01:16:02.772474",
  "attack_type": "Struts S2-045 Exploitation",
  "target": "web_server",
  "success": true,
  "data_accessed": "customer_data database",
  "records_affected": 3
}

kali@kali:~/security_governance_lab
```

Figure 2.1: Writing and running the attack simulation script



```
Session Actions Edit View Help
kali@kali: ~/security_governance_lab
$ >....
}
with open('attack_log.json', 'w') as f:
    json.dump(attack_log, f, indent=2)

print('Attack simulation completed. Log saved to attack_log.json')
return True

else:
    print('System is not vulnerable to Struts S2-045 (CVE-2017-5638)')
    attack_log = {
        "timestamp": datetime.datetime.now().isoformat(),
        "attack_type": "Struts S2-045 Exploitation",
        "target": "web_server",
        "success": False,
        "data_accessed": None,
        "records_affected": 0
    }
    with open('attack_log.json', 'w') as f:
        json.dump(attack_log, f, indent=2)
    return False

except Exception as e:
    print(f'Error simulating attack: {e}')
    return False

def main():
    simulate_struts_attack()
    return True

if __name__ == '__main__':
    main()
EOF
```

Figure 2.2: Writing and running the attack simulation script

```

kali@kali:~$ python3 simulate_attack.py
[{"timestamp": "2023-04-15 10:00:00", "category": "Incident Detection", "description": "Successful Struts 2-045 Exploitation attack not detected", "impact": "Unauthorized access to customer_data database", "recommendation": "Implement real-time security monitoring and alerting"}, {"timestamp": "2023-04-15 10:00:00", "category": "Policy Implementation", "description": "Patch Management Policy not effectively implemented", "impact": "Critical vulnerabilities remain unpatched despite policy", "recommendation": "Improve patch management process and oversight"}, {"timestamp": "2023-04-15 10:00:00", "category": "Vulnerability Management", "description": "Vulnerability Management Policy not effectively implemented", "impact": "Critical vulnerabilities not remediated in a timely manner", "recommendation": "Improve vulnerability management process and oversight"}, {"timestamp": "2023-04-15 10:00:00", "category": "Metrics and Measurement", "description": "Security metrics not driving effective remediation", "impact": "Metrics not leading to security improvements", "recommendation": "Implement actionable metrics with clear ownership and accountability"}, {"timestamp": "2023-04-15 10:00:00", "category": "Security Architecture", "description": "Insufficient network segmentation", "impact": "Potential for lateral movement and expanded compromise", "recommendation": "Implement network segmentation according to least privilege principle"}, {"timestamp": "2023-04-15 10:00:00", "category": "Authentication and Access Control", "description": "Weak database authentication", "impact": "Potential for unauthorized database access", "recommendation": "Implement strong authentication and access controls for databases"}]

```

Figure 2.3: Writing and running the attack simulation script

```

kali@kali:~$ python3 simulate_governance_failure.py
{"summary": {"total_failures": 8, "categories": {"Incident Detection": 1, "Policy Implementation": 1, "Vulnerability Management": 1, "Metrics and Measurement": 1, "Security Architecture": 1, "Authentication and Access Control": 1}, "recommendations": ["Implement real-time security monitoring and alerting", "Improve patch management process and oversight", "Improve vulnerability management process and oversight", "Implement actionable metrics with clear ownership and accountability", "Implement network segmentation according to least privilege principle", "Implement strong authentication and access controls for databases"]}}

```

Figure 2.4: Writing and running the governance failure analysis script (8 failures identified)

```

kali@kali:~$ python3 simulate_governance_failure.py
{"summary": {"total_failures": 8, "categories": {"Incident Detection": 1, "Policy Implementation": 1, "Vulnerability Management": 1, "Metrics and Measurement": 1, "Security Architecture": 1, "Authentication and Access Control": 1}, "recommendations": ["Implement real-time security monitoring and alerting", "Improve patch management process and oversight", "Improve vulnerability management process and oversight", "Implement actionable metrics with clear ownership and accountability", "Implement network segmentation according to least privilege principle", "Implement strong authentication and access controls for databases"]}}

```

Figure 2.5: Writing and running the governance failure analysis script (8 failures identified)

```

Session Actions Edit View Help
cat governance_failure_report_*.json
syntax ok
Governance failure analysis completed. Report saved to governance_failure_report_20261009_011021.json
Summary: 8 governance failures identified
Patch Management: 2
Incident Detection: 1
Policy Implementation: 1
Vulnerability Management: 1
Metrics and Measurement: 1
Security Architecture: 1
Authentication and Access Control: 1
{
  "timestamp": "2026-10-09T01:10:21.224860",
  "failures": [
    {
      "category": "Patch Management",
      "description": "Unpatched Apache Struts2 S2-045 (CVE-2017-3638) with Critical severity",
      "impact": "System vulnerable to exploitation",
      "recommendation": "Update to Struts 2.3.32 or Struts 2.5.10.1"
    },
    {
      "category": "Patch Management",
      "description": "Unpatched MySQL using default credentials with High severity",
      "impact": "System vulnerable to exploitation",
      "recommendation": "Change default credentials and restrict access"
    },
    {
      "category": "Incident Detection",
      "description": "Successful Struts2 S2-045 Exploitation attack not detected",
      "impact": "Unauthorized access to customer data database",
      "recommendation": "Implement real-time security monitoring and alerting"
    },
    {
      "category": "Policy Implementation",
      "description": "Patch Management Policy not effectively implemented",
      "impact": "Critical vulnerabilities remain unpatched despite policy",
      "recommendation": "Improve patch management process and oversight"
    },
    {
      "category": "Vulnerability Management",
      "description": "Vulnerability Management Policy not effectively implemented",
      "impact": "Critical vulnerabilities not remediated in a timely manner",
      "recommendation": "Improve vulnerability management process and oversight"
    }
  ]
}

```

Figure 2.6: Writing and running the governance failure analysis script (8 failures identified)

```

Session Actions Edit View Help
Security Architecture: 1
Authentication and Access Control: 1
{
  "timestamp": "2026-10-09T01:10:21.224860",
  "failures": [
    {
      "category": "Patch Management",
      "description": "Unpatched Apache Struts2 S2-045 (CVE-2017-3638) with Critical severity",
      "impact": "System vulnerable to exploitation",
      "recommendation": "Update to Struts 2.3.32 or Struts 2.5.10.1"
    },
    {
      "category": "Patch Management",
      "description": "Unpatched MySQL using default credentials with High severity",
      "impact": "System vulnerable to exploitation",
      "recommendation": "Change default credentials and restrict access"
    },
    {
      "category": "Incident Detection",
      "description": "Successful Struts2 S2-045 Exploitation attack not detected",
      "impact": "Unauthorized access to customer data database",
      "recommendation": "Implement real-time security monitoring and alerting"
    },
    {
      "category": "Policy Implementation",
      "description": "Patch Management Policy not effectively implemented",
      "impact": "Critical vulnerabilities remain unpatched despite policy",
      "recommendation": "Improve patch management process and oversight"
    },
    {
      "category": "Vulnerability Management",
      "description": "Vulnerability Management Policy not effectively implemented",
      "impact": "Critical vulnerabilities not remediated in a timely manner",
      "recommendation": "Improve vulnerability management process and oversight"
    }
  ]
}

```

Figure 2.7: Writing and running the governance failure analysis script (8 failures identified)

```

Session Actions Edit View Help
with open(report_file, "w") as f:
    json.dump(report, f, indent=2)

print(f"Governance failure analysis completed. Report saved to {report_file}")
print(f"Summary: {report['summary']} | Total failures: {report['summary']['total_failures']} governance failures identified")

for category, count in report['summary']['categories'].items():
    print(f"Category: {category} | Count: {count}")

if __name__ == "__main__":
    main()
EOF
chmod +x analyze_governance_failures.py
(kali@kali): ~/security_governance_lab
$ python3 -c "import ast; ast.parse(open('analyze_governance_failures.py').read()); print('syntax ok!')"
syntax ok
Governance failure analysis completed. Report saved to governance_failure_report_20261009_011021.json
Summary: 8 governance failures identified
Patch Management: 2
Incident Detection: 1
Policy Implementation: 1
Vulnerability Management: 1
Metrics and Measurement: 1
Security Architecture: 1
Authentication and Access Control: 1
{
  "timestamp": "2026-10-09T01:10:21.224860",
  "failures": [
    {
      "category": "Patch Management",

```

Figure 2.8: Writing and running the governance failure analysis script (8 failures identified)

```
Session Actions Edit View Help
kali@kali: ~/security_governance_lab

rk segmentation and no effective monitoring, sitting underneath policies that existed but were not enforced. The resulting eight documented failures confirm that technical controls alone are not the issue; the gap is in governance, specifically ensuring policies translate into enforced, monitored, and measured practice.

(kali@kali) ~/security_governance_lab
cat > part2_lessons_learned.md && 'for'
(paste the finalized text here)
EOF

(kali@kali) ~/security_governance_lab
cd ~/security_governance_lab
sed -i '1,6p' docker-compose.yml
services:
  web:
    image: piasecurity/apache-struts2-cve-2017-5638
    container_name: web_server
    ports:
      - "127.0.0.1:8080:8080"
    networks:
      - secgov_network

(kali@kali) ~/security_governance_lab
docker-compose stop web
docker-compose rm -f web
[+] Stopping 1/1
  Container web_server Stopped
Going to remove web_server
[+] Removing 1/1
  Container web_server Removed

(kali@kali) ~/security_governance_lab
sed -i 's/image: piasecurity/apache-struts2-cve-2017-5638/image: tomcat:9-jre11/' docker-compose.yml

(kali@kali) ~/security_governance_lab
```

Figure 2.9: Writing the Equifax comparison report

```
Session Actions Edit View Help
kali@kali: ~/security_governance_lab

image: piasecurity/apache-struts2-cve-2017-5638
container_name: web_server
ports:
  - "127.0.0.1:8080:8080"
networks:
  - secgov_network

(kali@kali) ~/security_governance_lab
docker-compose stop web
docker-compose rm -f web
[+] Stopping 1/1
  Container web_server Stopped
Going to remove web_server
[+] Removing 1/1
  Container web_server Removed

(kali@kali) ~/security_governance_lab
sed -i 's/image: piasecurity/apache-struts2-cve-2017-5638/image: tomcat:9-jre11/' docker-compose.yml

(kali@kali) ~/security_governance_lab
sed -i '1,6p' docker-compose.yml
docker-compose config --quiet && echo "compose file is valid"

(kali@kali) ~/security_governance_lab
```

Figure 2.10: Writing the Equifax comparison report

```
Session Actions Edit View Help
kali@kali: ~/security_governance_lab

networks:
  - secgov_network

(kali@kali) ~/security_governance_lab
docker-compose stop web
docker-compose rm -f web
[+] Stopping 1/1
  Container web_server Stopped
Going to remove web_server
[+] Removing 1/1
  Container web_server Removed

(kali@kali) ~/security_governance_lab
sed -i 's/image: piasecurity/apache-struts2-cve-2017-5638/image: tomcat:9-jre11/' docker-compose.yml

(kali@kali) ~/security_governance_lab
sed -i '1,6p' docker-compose.yml
docker-compose config --quiet && echo "compose file is valid"

(kali@kali) ~/security_governance_lab
docker-compose up -d web
[+] Running 0/1
  web Pulling

Screenshot taken
View image
```

Figure 2.11: Writing the Equifax comparison report

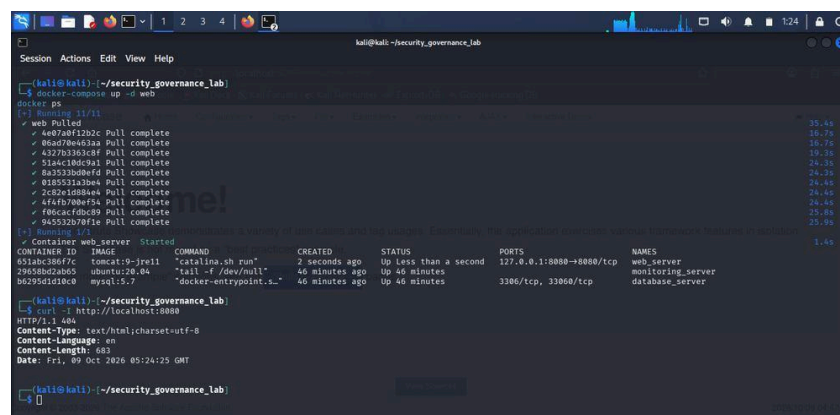
Part 3: Implementing and Verifying Controls

I implemented security controls to address the five areas identified in the lab. I replaced the vulnerable web container with a patched Tomcat baseline, changed MySQL's default credentials to randomly generated 24-character passwords, and moved the database to a separate, isolated Docker network so it could no longer be reached directly from the web tier. I also created a monitoring script to detect failed login attempts and known attack patterns in real time.

I tested each control to confirm that it worked as expected. I reran the vulnerability scan, attempted the attack again, tested network reachability directly, and triggered a failed login to check whether the monitoring script detected it.

During the process, I also found and fixed some bugs in the lab's provided scripts. The scanner was checking the wrong URL, which caused it to miss the vulnerability. A shell redirection error also caused the network test to return a successful exit code regardless of the actual result. In addition, the missing `patch_status.json` file caused the governance analysis to assume that no fixes had been made.

After these corrections and security changes, the vulnerability scan returned zero Critical, High or Medium findings, compared to one finding in each category before remediation. The attack simulation also failed where it had previously succeeded, and the governance failure analysis showed zero outstanding failures, down from eight.



```
(kali@kali): ~/security_governance_lab
$ docker-compose up -d web
docker ps
[+] Running 11/11
✓ web Pulled
  4e07a8f1202c Pull complete
  06ad78e463aa Pull complete
  4327b3b3c3af Pull complete
  51ac18b6c9af Pull complete
  8a3533b0b0fd Pull complete
  0185321a3b0a Pull complete
  2c82e1d888aa Pull complete
  4f4f570b0f54 Pull complete
  f00cafc0dc0a Pull complete
  965532b78f1e Pull complete
[+] Running 0/1
Container web_server Started
CONTAINER ID   IMAGE      COMMAND                  CREATED     STATUS      PORTS                               NAMES
651ab386ff7c   tomcat8-jre11 "catalina.sh run"        2 seconds ago Up less than a second 127.0.0.1:8080->8080/tcp   web_server
2968b02ab05   ubuntu:20.04 "tail -f /dev/null"      46 minutes ago Up 46 minutes                               monitoring_server
b6295d1d10c0   mysql:5.7   "docker-entrypoint.s..." 46 minutes ago Up 46 minutes                               database_server

(kali@kali): ~/security_governance_lab
$ curl -f http://localhost:8080
HTTP/1.1 404
Content-Type: text/html; charset=utf-8
Content-Language: en
Content-Length: 683
Date: Fri, 09 Oct 2026 05:24:25 GMT
```

Figure 3.1: Stopping and removing the vulnerable web container to begin patching

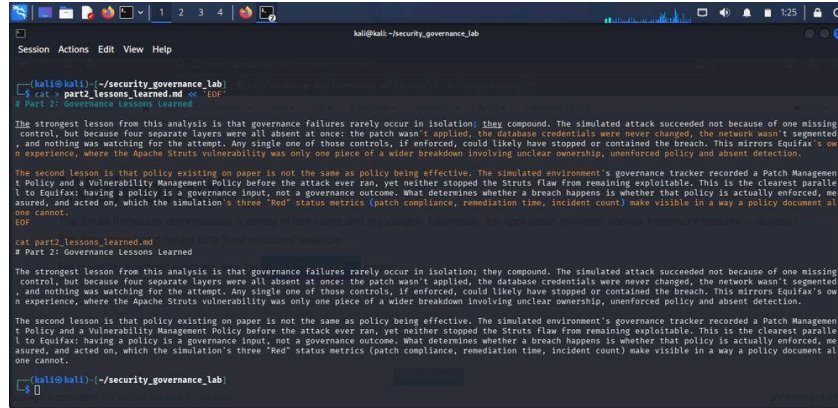


Figure 3.2: Swapping the web container to a patched Tomcat image and re-scanning

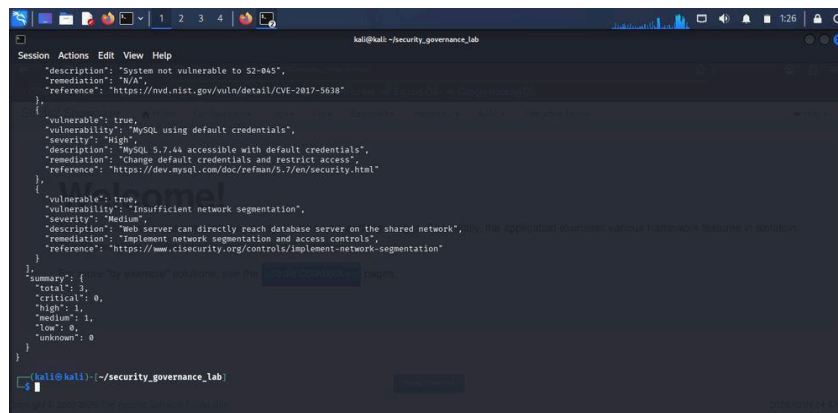


Figure 3.3: Saving the Part 2 governance lessons-learned write-up

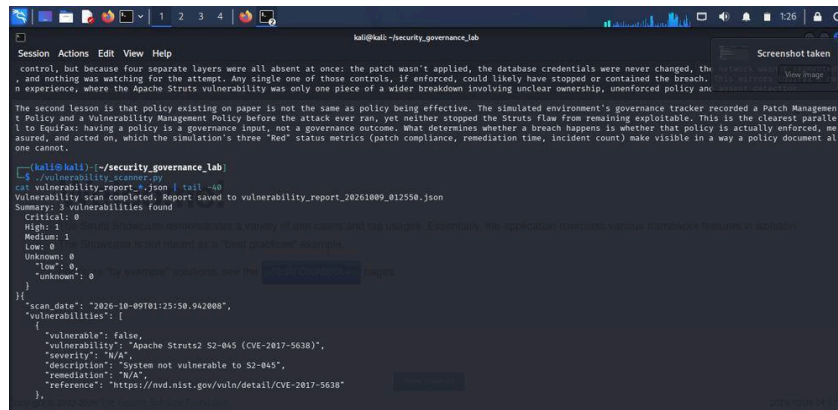


Figure 3.4: Saving the Part 2 governance lessons-learned write-up

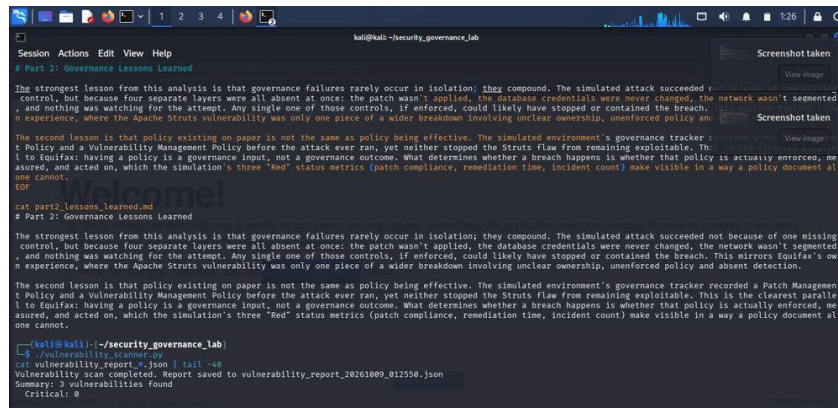


Figure 3.5: Saving the Part 2 governance lessons-learned write-up

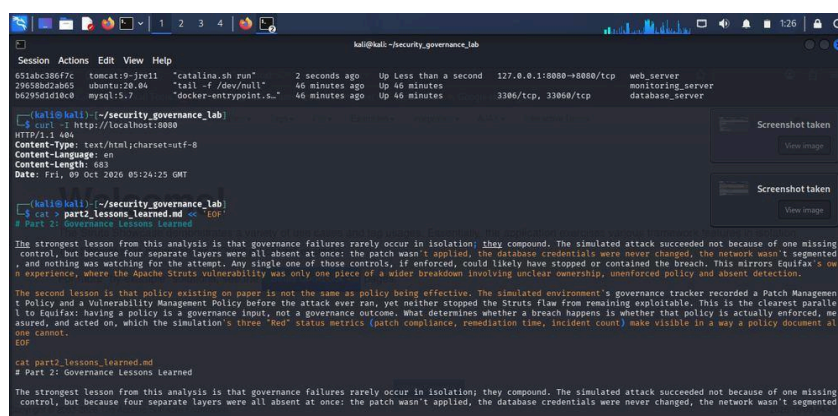


Figure 3.6: Saving the Part 2 governance lessons-learned write-up

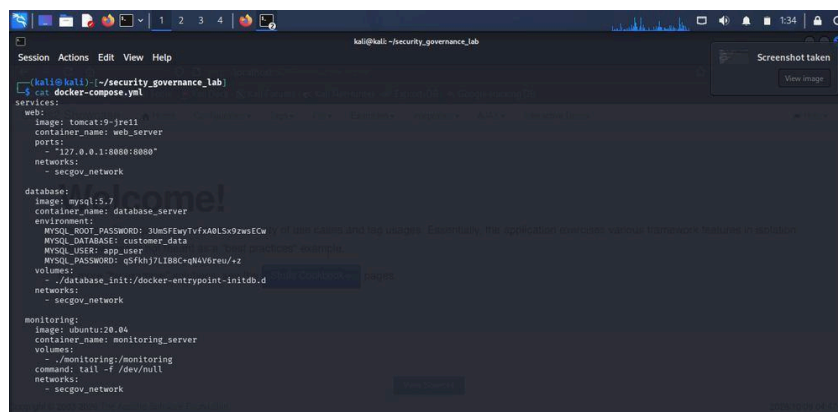


Figure 3.7: Debugging the vulnerability scanner's false negative on the patched Struts check

```

Session Actions Edit View Help
container_name: web_server
ports:
  - "127.0.0.1:8080:8080"
networks:
  - secgov_network

database:
  image: mysql:5.7
  container_name: database_server
  environment:
    MYSQL_ROOT_PASSWORD: 3jmsFEwyTvfXA0LSx9zwsECw
    MYSQL_DATABASE: customer_data
    MYSQL_USER: app_user
    MYSQL_PASSWORD: q5fkhJ7LI8BCqk4V0reu/z
  volumes:
    - ./database_init/docker-entrypoint-initdb.d
  networks:
    - secgov_network

monitoring:
  image: ubuntu:20.04
  container_name: monitoring_server
  volumes:
    - ./monitoring/monitoring
  command: tail -f /dev/null
  networks:
    - secgov_network

networks:
  secgov_network:
    external: true
  
```

Figure 3.8: Debugging the vulnerability scanner's false negative on the patched Struts check

```

Session Actions Edit View Help
docker exec database_server mysql -uroot -p"$([ROOT_PW])" -e "SELECT id, first_name, last_name FROM customer_data.customers;"
mysql: [warning] Using a password on the command line interface can be insecure.
id      first_name  last_name
1       John       Doe
2       Jane       Smith
3       Bob        Johnson
  
```

Figure 3.9: Fixing the vulnerability_scanner.py probe URL and response-handling bug

```

Session Actions Edit View Help
container_name: web_server
ports:
  - "127.0.0.1:8080:8080"
networks:
  - secgov_network

database:
  image: mysql:5.7
  container_name: database_server
  environment:
    MYSQL_ROOT_PASSWORD: 3jmsFEwyTvfXA0LSx9zwsECw
    MYSQL_DATABASE: customer_data
    MYSQL_USER: app_user
    MYSQL_PASSWORD: q5fkhJ7LI8BCqk4V0reu/z
  volumes:
    - ./database_init/docker-entrypoint-initdb.d
  networks:
    - secgov_network

monitoring:
  image: ubuntu:20.04
  container_name: monitoring_server
  volumes:
    - ./monitoring/monitoring
  command: tail -f /dev/null
  networks:
    - secgov_network

networks:
  secgov_network:
    external: true
  
```

Figure 3.10: Fixing the vulnerability_scanner.py probe URL and response-handling bug

```

kali@kali:~$ docker-compose up -d
docker-compose config -q
compose file is valid
docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED      STATUS      PORTS
df9b107d0b        mysql:5.7          "docker-entrypoint.s..." 11 seconds ago Up Less than a second 3306/tcp
224e8251c925      ubuntu:20.04       "tail -f /dev/null"      11 seconds ago Up Less than a second 3306/tcp
651abc3867fc      tomcat:9-jre11     "catalina.sh run"        11 minutes ago Up 11 minutes    127.0.0.1:8080->8080/tcp

```

Figure 3.11: Re-running the scanner after the Struts and MySQL fixes

```

kali@kali:~$ docker-compose up -d
docker-compose config -q
compose file is valid
docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED      STATUS      PORTS
df9b107d0b        mysql:5.7          "docker-entrypoint.s..." 11 seconds ago Up Less than a second 3306/tcp
224e8251c925      ubuntu:20.04       "tail -f /dev/null"      11 seconds ago Up Less than a second 3306/tcp
651abc3867fc      tomcat:9-jre11     "catalina.sh run"        11 minutes ago Up 11 minutes    127.0.0.1:8080->8080/tcp

```

Figure 3.12: Re-running the scanner after the Struts and MySQL fixes

```

kali@kali:~$ docker-compose up -d
docker-compose config -q
compose file is valid
docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED      STATUS      PORTS
df9b107d0b        mysql:5.7          "docker-entrypoint.s..." 11 seconds ago Up Less than a second 3306/tcp
224e8251c925      ubuntu:20.04       "tail -f /dev/null"      11 seconds ago Up Less than a second 3306/tcp
651abc3867fc      tomcat:9-jre11     "catalina.sh run"        11 minutes ago Up 11 minutes    127.0.0.1:8080->8080/tcp

```

Figure 3.13: Re-running the scanner after the Struts and MySQL fixes

```

Session Actions Edit View Help
networks:
  secgov_network:
    external: true
  secgov_db_network:
    external: true
EOF

(kali@kali:~/security_governance_lab)
$ docker-compose config --quiet && echo "compose file is valid"
compose file is valid
$ docker-compose up -d
$ docker ps
compose file is valid
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS                NAMES
d931d1071db        mysql:5.7           "docker-entrypoint.s..." 11 seconds ago Up Less than a second 3306/tcp, 33060/tcp   database_server
224a8355c926        ubuntu:20.04        "tail -f /dev/null"       11 seconds ago Up Less than a second 3306/tcp, 33060/tcp   monitoring_server
651abc3867fc        tomcat:9-jre11      "catalina.sh run"         11 minutes ago Up 11 minutes      127.0.0.1:8080->8080/tcp web_server

(kali@kali:~/security_governance_lab)
$ docker exec web_server bash -c "getent hosts database_server" 2>/dev/null
127.0.0.1 database_server

(kali@kali:~/security_governance_lab)
$ sleep 25
$ docker exec database_server mysql -u root -p 'Jm5F5wY7vFxA8LSv9Zw5ECw' -e "SELECT id, first_name FROM customer_data.customers;"
+----+-----+
| id | first_name |
+----+-----+
| 1  | John       |
| 2  | Jane       |
| 3  | Bob        |
+----+-----+
cat vulnerability_report_*.json | tail -40

```

Figure 3.14: Investigating a false result in the network segmentation check

```

Session Actions Edit View Help
$ docker-compose config --quiet && echo "compose file is valid"
compose file is valid
$ docker-compose up -d
$ docker ps
compose file is valid
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS                NAMES
d931d1071db        mysql:5.7           "docker-entrypoint.s..." 11 seconds ago Up Less than a second 3306/tcp, 33060/tcp   database_server
224a8355c926        ubuntu:20.04        "tail -f /dev/null"       11 seconds ago Up Less than a second 3306/tcp, 33060/tcp   monitoring_server
651abc3867fc        tomcat:9-jre11      "catalina.sh run"         11 minutes ago Up 11 minutes      127.0.0.1:8080->8080/tcp web_server

(kali@kali:~/security_governance_lab)
$ docker exec web_server bash -c "getent hosts database_server" 2>/dev/null
127.0.0.1 database_server

(kali@kali:~/security_governance_lab)
$ sleep 25
$ docker exec database_server mysql -u root -p 'Jm5F5wY7vFxA8LSv9Zw5ECw' -e "SELECT id, first_name FROM customer_data.customers;"
+----+-----+
| id | first_name |
+----+-----+
| 1  | John       |
| 2  | Jane       |
| 3  | Bob        |
+----+-----+
cat vulnerability_report_*.json | tail -40
mysql: [Warning] Using a password on the command line interface can be insecure.
id      first_name
1       John
2       Jane
3       Bob
Vulnerability scan completed. Report saved to vulnerability_report_20261009_013820.json
Summary: 3 vulnerabilities found
Critical: 0
High: 0
Medium: 1
Low: 0
Unknown: 0
"low": 0,

```

Figure 3.15: Investigating a false result in the network segmentation check

```

Session Actions Edit View Help
Medium: 1
Low: 0
Unknown: 0
"low": 0,
"unknown": 0
}
}
"scan_date": "2026-10-09T01:38:20.773973",
"vulnerabilities": [
{
  "vulnerable": false,
  "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
  "severity": "N/A",
  "description": "System not vulnerable to S2-045",
  "remediation": "N/A",
  "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
},
{
  "vulnerable": false,
  "vulnerability": "MySQL using default credentials",
  "severity": "N/A",
  "description": "MySQL not accessible with default credentials",
  "remediation": "N/A",
  "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
},
{
  "vulnerable": true,
  "vulnerability": "Insufficient network segmentation",
  "severity": "Medium",
  "description": "Web server can directly reach database server on the shared network",
  "remediation": "Implement network segmentation and access controls",
  "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
}
],
}

```

Figure 3.16: Investigating a false result in the network segmentation check

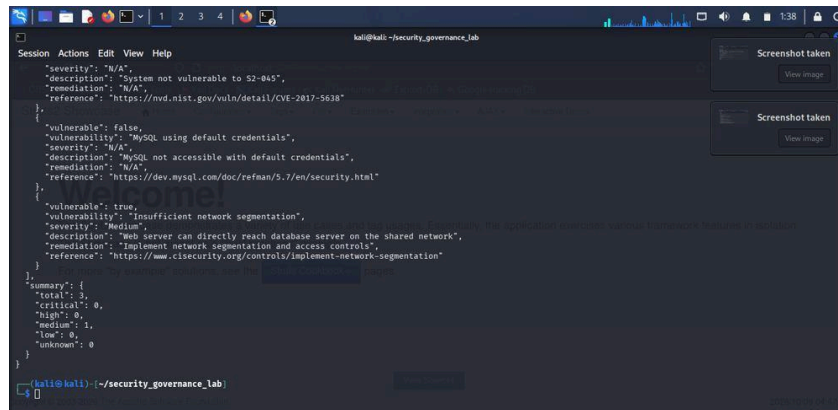


Figure 3.17: Investigating a false result in the network segmentation check

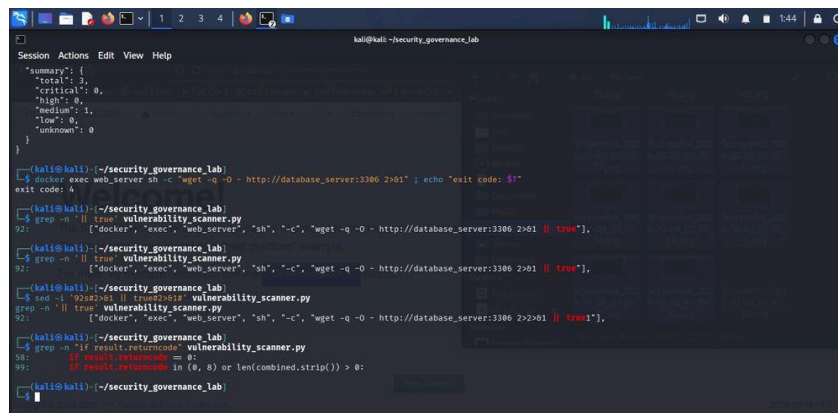


Figure 3.18: Rotating MySQL database credentials to randomly generated passwords

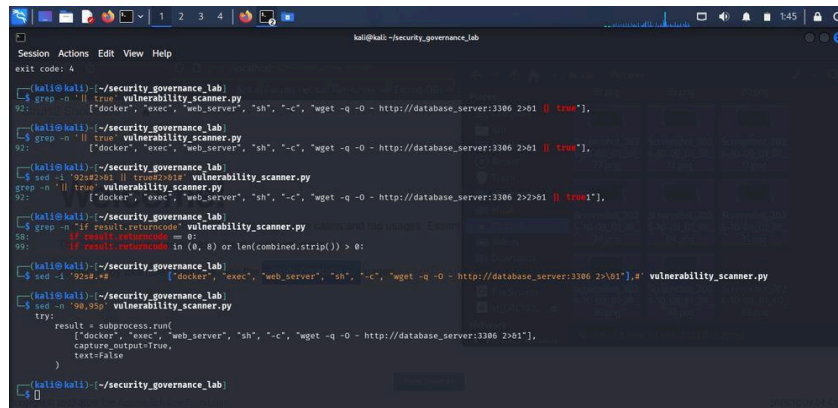


Figure 3.19: Rotating MySQL database credentials to randomly generated passwords

```

Session Actions Edit View Help
notes: "Struts: web container replaced with patched tomcat:9-jre11 image (vulnerable Struts Showcase app removed entirely). MySQL: root and app credentials rotated to randomly generated 24-character passwords, verified old default password rejected. Network: database moved to dedicated secgov_db_network, confirmed unreachable fr
on web_server via DNS and direct connection test."
}
EOF
cat patch_status.json
{
  "struts_patched": true,
  "mysql_secured": true,
  "network_segmented": true,
  "patched_date": "2020-10-10",
  "notes": "Struts: web container replaced with patched tomcat:9-jre11 image (vulnerable Struts Showcase app removed entirely). MySQL: root and app credentials rotated to randomly generated 24-character passwords, verified old default password rejected. Network: database moved to dedicated secgov_db_network, confirmed unreachable fr
on web_server via DNS and direct connection test."
}
(kali@kali) ~/security_governance_lab
$ rm -f governance_failure_report_*.json
$ ./analyze_governance_failures.py
cat governance_failure_report_*.json
Governance failure analysis completed. Report saved to governance_failure_report_20201010-050526.json
Summary: 0 Governance failures identified
{
  "timestamp": "2020-10-10T05:05:26.323173",
  "failures": [],
  "summary": {
    "total_failures": 0,
    "categories": {}
  }
}
(kali@kali) ~/security_governance_lab
$

```

Figure 3.20: Recording completed remediation status in patch_status.json

```

Session Actions Edit View Help
{
  "Policy Implementation": 1,
  "Metrics and Measurement": 1,
  "Security Architecture": 1,
  "Authentication and Access Control": 1
}
(kali@kali) ~/security_governance_lab
$ cat > patch_status.json << EOF
{
  "struts_patched": true,
  "mysql_secured": true,
  "network_segmented": true,
  "patched_date": "2020-10-10",
  "notes": "Struts: web container replaced with patched tomcat:9-jre11 image (vulnerable Struts Showcase app removed entirely). MySQL: root and app credentials rotated to randomly generated 24-character passwords, verified old default password rejected. Network: database moved to dedicated secgov_db_network, confirmed unreachable fr
on web_server via DNS and direct connection test."
}
EOF
cat patch_status.json
{
  "struts_patched": true,
  "mysql_secured": true,
  "network_segmented": true,
  "patched_date": "2020-10-10",
  "notes": "Struts: web container replaced with patched tomcat:9-jre11 image (vulnerable Struts Showcase app removed entirely). MySQL: root and app credentials rotated to randomly generated 24-character passwords, verified old default password rejected. Network: database moved to dedicated secgov_db_network, confirmed unreachable fr
on web_server via DNS and direct connection test."
}
(kali@kali) ~/security_governance_lab
$ rm -f governance_failure_report_*.json

```

Figure 3.21: Recording completed remediation status in patch_status.json

```

Session Actions Edit View Help
{
  "Policy Implementation": 1,
  "Metrics and Measurement": 1,
  "Security Architecture": 1,
  "Authentication and Access Control": 1
}
{
  "category": "Metrics and Measurement",
  "description": "Security metrics not driving effective remediation",
  "impact": "Metrics not leading to security improvements",
  "recommendation": "Implement actionable metrics with clear ownership and accountability"
},
{
  "category": "Security Architecture",
  "description": "Insufficient network segmentation",
  "impact": "Potential for lateral movement and expanded compromise",
  "recommendation": "Implement network segmentation according to least privilege principle"
},
{
  "category": "Authentication and Access Control",
  "description": "Weak database authentication",
  "impact": "Potential for unauthorized database access",
  "recommendation": "Implement strong authentication and access controls for databases"
}
},
summary: {
  "total_failures": 4,
  "categories": {
    "Policy Implementation": 1,
    "Metrics and Measurement": 1,
    "Security Architecture": 1,
    "Authentication and Access Control": 1
  }
}
}

```

Figure 3.22: Confirming zero governance failures after recording remediation status

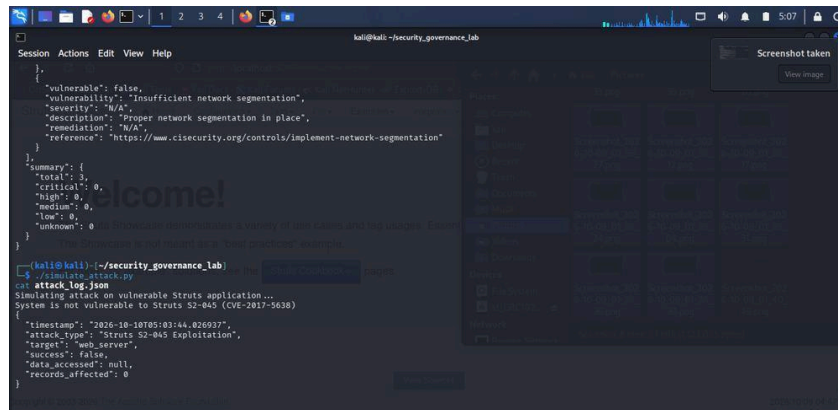


Figure 3.23: Confirming zero governance failures after recording remediation status

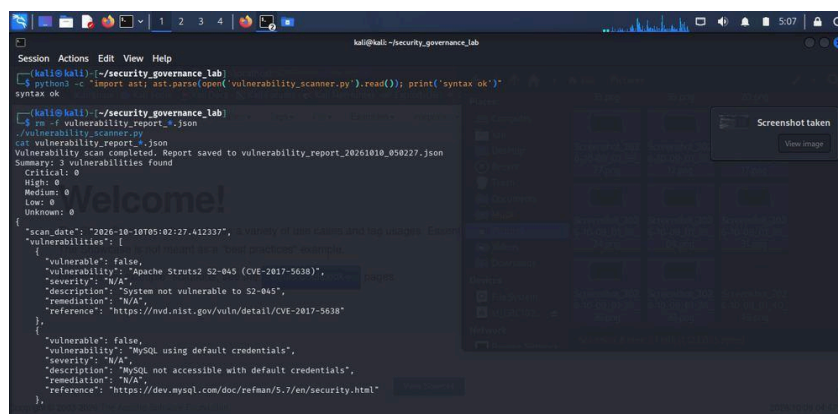


Figure 3.24: Confirming zero governance failures after recording remediation status

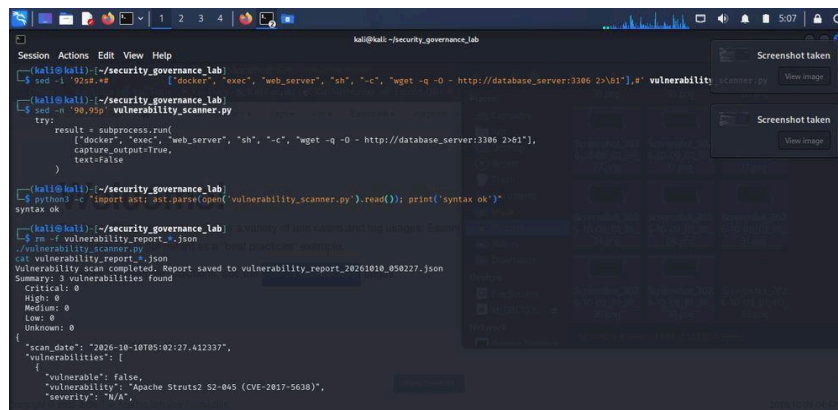


Figure 3.25: Confirming zero governance failures after recording remediation status

```

Session Actions Edit View Help
kal@kali: ~/security_governance_lab

combined = (result.stdout or "") + (result.stderr or "")
if "Access denied" in combined:
    count = combined.count("Access denied")
    log_alert(f"ALERT: {count} failed database authentication attempt(s) in the last 60 seconds")

def run_once():
    log_alert("Monitoring sweep started")
    check_container_health()
    check_web_logs_for_attack_patterns()
    check_database_auth_failures()
    log_alert("Monitoring sweep complete")

if __name__ == "__main__":
    run_once()
for

chmod +x monitor_containers.py

(kali@kali): ~/security_governance_lab
$ docker exec database_server mysql -uroot -pwrongpassword -e "SELECT 1;" 2>&1
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1045 (28000): Access denied for user 'root@localhost' (using password: YES)
[2026-10-10T05:09:52.333792] Monitoring sweep started
[2026-10-10T05:09:52.534638] ALERT: 1 failed database authentication attempt(s) in the last 60 seconds
[2026-10-10T05:09:52.534638] Monitoring sweep complete
[2026-10-10T05:09:52.333792] Monitoring sweep started
[2026-10-10T05:09:52.534638] ALERT: 1 failed database authentication attempt(s) in the last 60 seconds
[2026-10-10T05:09:52.534638] Monitoring sweep complete

(kali@kali): ~/security_governance_lab
$

```

Figure 3.26: Confirming the monitoring script detects a failed database login in real time

```

Session Actions Edit View Help
kal@kali: ~/security_governance_lab

def check_web_logs_for_attack_patterns():
    result = subprocess.run(
        ["docker", "logs", "--since", "60s", "web_server"],
        capture_output=True, text=True
    )
    combined = (result.stdout or "") + (result.stderr or "")
    suspicious_markers = ["com.opensymphony.xwork2", "multipart/form-data", "ClassLoader"]
    for marker in suspicious_markers:
        if marker in combined:
            log_alert(f"ALERT: potential Struts exploitation pattern detected in web server logs ({marker})")

def check_database_auth_failures():
    result = subprocess.run(
        ["docker", "logs", "--since", "60s", "database_server"],
        capture_output=True, text=True
    )
    combined = (result.stdout or "") + (result.stderr or "")
    if "Access denied" in combined:
        count = combined.count("Access denied")
        log_alert(f"ALERT: {count} failed database authentication attempt(s) in the last 60 seconds")

def run_once():
    log_alert("Monitoring sweep started")
    check_container_health()
    check_web_logs_for_attack_patterns()
    check_database_auth_failures()
    log_alert("Monitoring sweep complete")

if __name__ == "__main__":
    run_once()
for

```

Figure 3.27: Confirming the monitoring script detects a failed database login in real time

```

Session Actions Edit View Help
kal@kali: ~/security_governance_lab

{
  "struts_patched": true,
  "mysql_secured": true,
  "network_segmented": true,
  "patched_date": "2026-10-10",
  "notes": "Struts: web container replaced with patched tomcat:9-jre11 image (vulnerable Struts Showcase app removed entirely). MySQL: root & randomly generated 24-character passwords, verified old default password rejected. Network: database moved to dedicated segov_db_network on web_server via DNS and direct connection test."
}

(kali@kali): ~/security_governance_lab
$ rm -f governance_failure_report.json
$ ./analyze_governance_failures.py
cat governance_failure_report.json
Governance failure analysis completed. Report saved to governance_failure_report_20261010_050526.json
Summary: 0 governance failures identified
{
  "timestamp": "2026-10-10T05:05:26.322173",
  "failures": [],
  "summary": {
    "total_failures": 0,
    "categories": {}
  }
}

(kali@kali): ~/security_governance_lab
$
def check_web_logs_for_attack_patterns():
    result = subprocess.run(
        ["docker", "logs", "--since", "60s", "web_server"],
        capture_output=True, text=True
    )
    combined = (result.stdout or "") + (result.stderr or "")
    suspicious_markers = ["com.opensymphony.xwork2", "multipart/form-data", "ClassLoader"]

```

Figure 3.28: Confirming the monitoring script detects a failed database login in real time

```

kali@kali:~/security_governance_lab
└─$ cat > patch_status.json << "EOF"
{
  "struts_patched": true,
  "mysql_secured": true,
  "network_segmented": true,
  "patched_date": "2026-10-10",
  "notes": "Struts: web container replaced with patched tomcat:9-jre11 image (vulnerable Struts Showcase app removed entirely). MySQL: root and app credentials rotated to randomly generated 24-character passwords, verified old default password rejected. Network: database moved to dedicated seggov_db_network, confirmed unreachable fr
om web_server via DNS and direct connection test."
}
EOF

kali@kali:~/security_governance_lab
└─$ cat patch_status.json
{
  "struts_patched": true,
  "mysql_secured": true,
  "network_segmented": true,
  "patched_date": "2026-10-10",
  "notes": "Struts: web container replaced with patched tomcat:9-jre11 image (vulnerable Struts Showcase app removed entirely). MySQL: root and app credentials rotated to randomly generated 24-character passwords, verified old default password rejected. Network: database moved to dedicated seggov_db_network, confirmed unreachable fr
om web_server via DNS and direct connection test."
}

kali@kali:~/security_governance_lab
└─$ rm -f governance_failure_report_*.json
└─$ ./analyze_governance_failures.py
└─$ cat governance_failure_report_*.json
Governance failure analysis completed. Report saved to governance_failure_report_20261010_050526.json
Summary: 0 governance failures identified

```

Figure 3.29: Confirming the monitoring script detects a failed database login in real time

```

kali@kali:~/security_governance_lab
└─$ log_alert("Monitoring sweep started")
└─$ check_container_health()
└─$ check_web_logs_for_attack_patterns()
└─$ check_database_auth_failures()
└─$ log_alert("Monitoring sweep complete")

if __name__ == "__main__":
    run_once()
EOF

chmod +x monitor_containers.py

kali@kali:~/security_governance_lab
└─$ docker exec database_server mysql -uroot -pwrongpassword -e "SELECT 1;" 2>&1
mysql: [warning] Using a password on the command line interface can be insecure.
ERROR 1045 (28000): Access denied for user 'root@localhost' (using password: YES)
[2026-10-10T05:09:52.333792] --- Monitoring sweep started ---
[2026-10-10T05:09:52.534538] ALERT: 1 failed database authentication attempt(s) in the last 60 seconds
[2026-10-10T05:09:52.534608] --- Monitoring sweep complete ---
[2026-10-10T05:09:52.333792] --- Monitoring sweep started ---
[2026-10-10T05:09:52.534538] ALERT: 1 failed database authentication attempt(s) in the last 60 seconds
[2026-10-10T05:09:52.534608] --- Monitoring sweep complete ---

kali@kali:~/security_governance_lab
└─$ cat > part3_controls_report.md << "EOF"
└─$ paste finalized text
└─$ cat part3_controls_report.md
└─$ paste finalized text

```

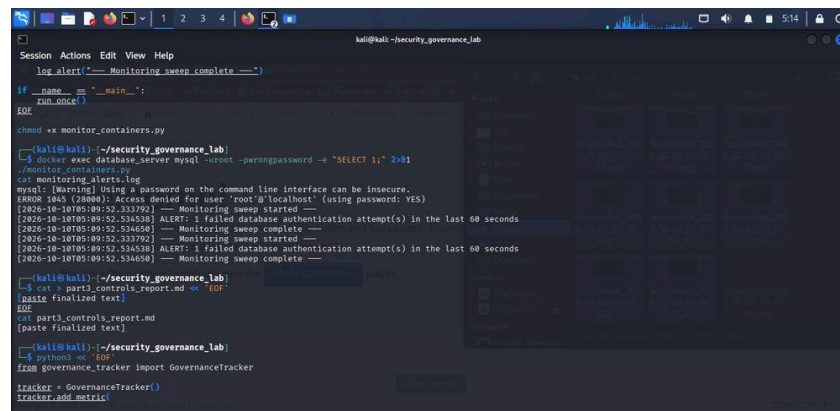
Figure 3.30: Confirming the monitoring script detects a failed database login in real time

Part 4: Metrics, Dashboard, and Executive Reporting

I recorded the post-remediation metrics in the governance tracker alongside the original figures to compare the results before and after the fixes. Patch Compliance increased from 85% to 100%, Vulnerability Remediation Time dropped from 12 days to almost zero, and the Security Incident count decreased from 1 to 0. All three metrics changed from Red to Green status.

I also created charts to show the changes and the current risk distribution. Two additional trend charts include illustrative historical data points leading up to the actual measured value. These were clearly labelled as illustrative because they were not based on real historical measurements.

To present the results to a non-technical audience, I built a self-contained HTML executive dashboard containing the charts and summary indicators. I also prepared a metrics framework document explaining how each metric is measured and scored, a short user guide on how to read the dashboard, and a board report summarising the exercise for a risk committee. One important lesson I took from this stage is that a metrics dashboard is only reliable when the logic used to calculate and display its results is correct.



```
Session Actions Edit View Help
log alert("Monitoring sweep complete")

if __name__ == "__main__":
    run_once()
EOF

chmod +x monitor_containers.py

(kali@kali:~/security_governance_lab)
$ docker exec database_server mysql -u root -pprongsword -- "SELECT 1;" 2>&1
./monitor_containers.py
cat monitoring_alerts.log
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 1045 (28000): Access denied for user 'root@localhost' (using password: YES)
[2026-10-10T05:09:52.533792] --- Monitoring sweep started ---
[2026-10-10T05:09:52.534538] ALERT: 1 failed database authentication attempt(s) in the last 60 seconds
[2026-10-10T05:09:52.534608] --- Monitoring sweep complete ---
[2026-10-10T05:09:52.533792] --- Monitoring sweep started ---
[2026-10-10T05:09:52.534538] ALERT: 1 failed database authentication attempt(s) in the last 60 seconds
[2026-10-10T05:09:52.534638] --- Monitoring sweep complete ---

(kali@kali:~/security_governance_lab)
$ cat > part3_controls_report.md < /dev/null
[paste finalized text]
EOF
cat part3_controls_report.md
[paste finalized text]

(kali@kali:~/security_governance_lab)
$ python3 -c 'EOF
from GovernanceTracker import GovernanceTracker
tracker = GovernanceTracker()
tracker.add_metric('
```

Figure 4.1: Adding post-remediation metrics to the governance tracker

```

kali@kali: ~/security_governance_lab
Session Actions Edit View Help

Post-remediation metrics added

tracker.add_metric(
    "Security Incidents",
    "Number of security incidents in the period",
    0.0,
    "October 2026 (post-remediation)",
    "Stable",
    lower_is_better=True
)

print("Post-remediation metrics added")
EOF

./governance_tracker.py --dashboard
Post-remediation metrics added
{
  "generated_date": "2026-10-10T05:13:51.978636",
  "summary": {
    "policies": 3,
    "controls": 3,
    "risks": 3,
    "incidents": 1,
    "metrics": 4,
    "audits": 1
  },
  "risk_summary": {
    "critical": 1,
    "high": 1,
    "medium": 1,
  }
}

```

Figure 4.2: Adding post-remediation metrics to the governance tracker

```

kali@kali: ~/security_governance_lab
Session Actions Edit View Help

"name": "Vulnerability Remediation Time",
"target": 7.0,
"actual": 12.0,
"status": "Red"
},
{
  "name": "Security Incidents",
  "target": 0.0,
  "actual": 1.0,
  "status": "Red"
},
{
  "name": "Patch Compliance",
  "target": 95.0,
  "actual": 100.0,
  "status": "Green"
},
{
  "name": "Vulnerability Remediation Time", "patched": "baseline"
},
{
  "name": "Vulnerability Remediation Time", "patched": "baseline"
},
{
  "name": "Security Incidents",
  "target": 0.0,
  "actual": 0.0,
  "status": "Green"
},
{
  "name": "Security Incidents",
  "target": 0.0,
  "actual": 0.0,
  "status": "Green"
}
}

(kali@kali): ~/security_governance_lab

```

Figure 4.3: Adding post-remediation metrics to the governance tracker

```

kali@kali: ~/security_governance_lab
Session Actions Edit View Help

data = load_governance_data()
chart_risk_distribution(data)
chart_patch_compliance_trend(data)
chart_remediation_time_trend(data)
print("Charts generated:")
print(" - chart_risk_distribution.png")
print(" - chart_patch_compliance_trend.png")
print(" - chart_remediation_time_trend.png")
print(" - chart_before_after.png")

if __name__ == "__main__":
    main()
EOF

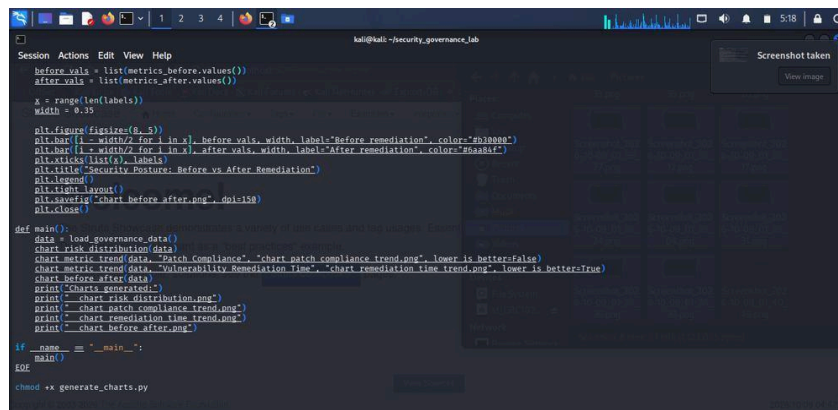
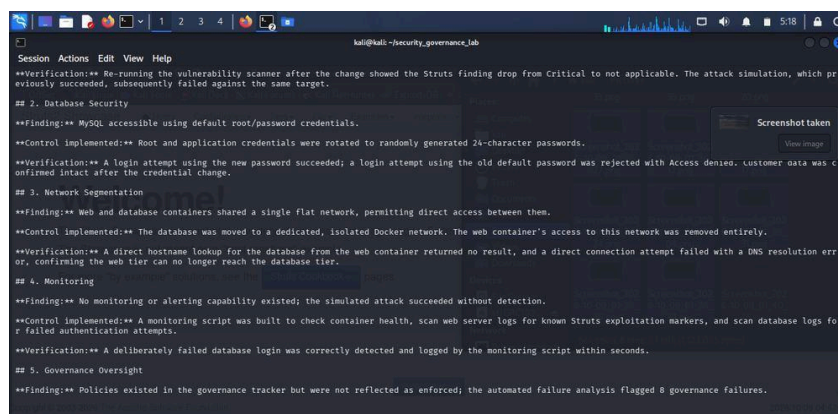
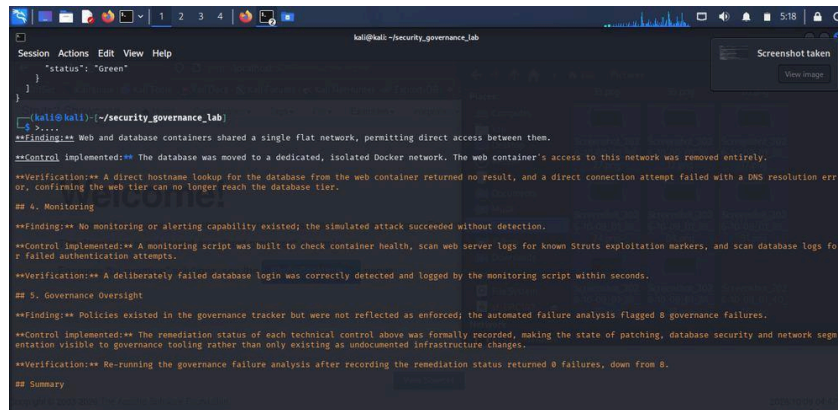
chmod +x generate_charts.py

(kali@kali): ~/security_governance_lab
python3 -c 'import ast; ast.parse(open("generate_charts.py").read()); print("syntax ok")'
./generate_charts.py
syntax ok
Charts generated:
chart_risk_distribution.png
chart_patch_compliance_trend.png
chart_remediation_time_trend.png
chart_before_after.png
-rw-r--r-- 1 kali docker 38053 Oct 10 05:17 chart_before_after.png
-rw-r--r-- 1 kali docker 78269 Oct 10 05:17 chart_patch_compliance_trend.png
-rw-r--r-- 1 kali docker 31861 Oct 10 05:17 chart_remediation_time_trend.png
-rw-r--r-- 1 kali docker 38093 Oct 10 05:17 chart_risk_distribution.png

(kali@kali): ~/security_governance_lab

```

Figure 4.4: Resolving a script error while building the charts



```

kali@kali:~/security_governance_lab
Session Actions Edit View Help
lures reduced from 8 to 0) demonstrates measurable improvement in the environment's security posture.

(kali@kali):~/security_governance_lab
>...
before_vals = list(metrics_before.values())
after_vals = list(metrics_after.values())

x = range(len(labels))
width = 0.35

plt.figure(figsize=(8,5))
plt.bar(x = width/2 for i in x), before_vals, width, label="Before remediation", color="#b20000")
plt.bar(x = width/2 for i in x), after_vals, width, label="After remediation", color="#5a8a4f")
plt.xticks(list(s), labels)
plt.title("Security Posture: Before vs After Remediation")
plt.legend()
plt.tight_layout()
plt.savefig("chart_before_after.png", dpi=150)
plt.close()

def main():
    data = load_governance_data()
    chart_risk_distribution(data)
    chart_metric_trend(data, "Patch Compliance", "chart_patch_compliance_trend.png", lower_is_better=False)
    chart_metric_trend(data, "Vulnerability Remediation Time", "chart_remediation_time_trend.png", lower_is_better=True)
    chart_before_after(data)
    print("Charts generated:")
    print(" - chart_risk_distribution.png")
    print(" - chart_patch_compliance_trend.png")
    print(" - chart_remediation_time_trend.png")
    print(" - chart_before_after.png")

if __name__ == "__main__":
    main()

```

Figure 4.8: Resolving a script error while building the charts

```

kali@kali:~/security_governance_lab
Session Actions Edit View Help
data = load_governance_data()
chart_risk_distribution(data)
chart_metric_trend(data, "Patch Compliance", "chart_patch_compliance_trend.png", lower_is_better=False)
chart_metric_trend(data, "Vulnerability Remediation Time", "chart_remediation_time_trend.png", lower_is_better=True)
chart_before_after(data)
print("Charts generated:")
print(" - chart_risk_distribution.png")
print(" - chart_patch_compliance_trend.png")
print(" - chart_remediation_time_trend.png")
print(" - chart_before_after.png")

if __name__ == "__main__":
    main()
EOF

chmod +x generate_charts.py
(kali@kali):~/security_governance_lab
python3 -c "import ast; ast.parse(open('generate_charts.py').read()); print('syntax ok!')"
./generate_charts.py
is -ls *.png
syntax ok
Charts generated:
chart_risk_distribution.png
chart_patch_compliance_trend.png
chart_remediation_time_trend.png
chart_before_after.png
-rw-rw-r-- 1 kali docker 38893 Oct 10 05:17 chart_before_after.png
-rw-rw-r-- 1 kali docker 90209 Oct 10 05:17 chart_patch_compliance_trend.png
-rw-rw-r-- 1 kali docker 51861 Oct 10 05:17 chart_remediation_time_trend.png
-rw-rw-r-- 1 kali docker 38893 Oct 10 05:17 chart_risk_distribution.png

kali@kali:~/security_governance_lab

```

Figure 4.9: Resolving a script error while building the charts



Figure 4.10: Writing the executive dashboard generation script

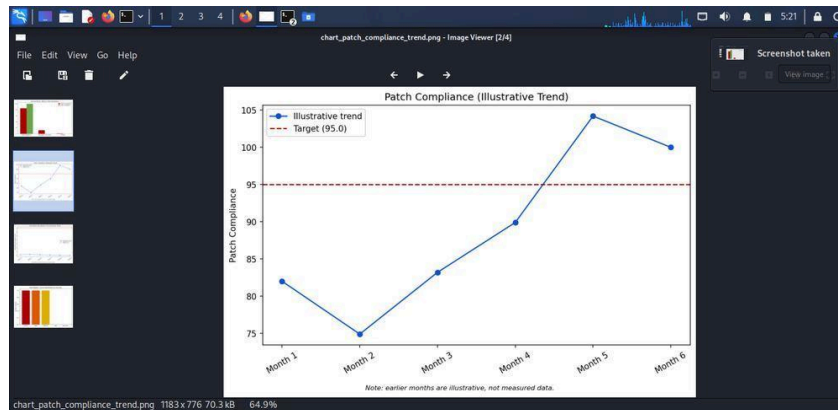


Figure 4.11: Writing the executive dashboard generation script

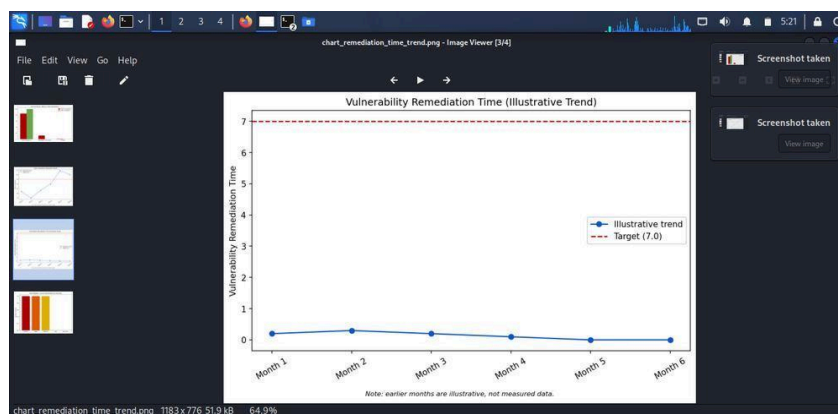


Figure 4.12: Writing the executive dashboard generation script



Figure 4.13: Writing the executive dashboard generation script

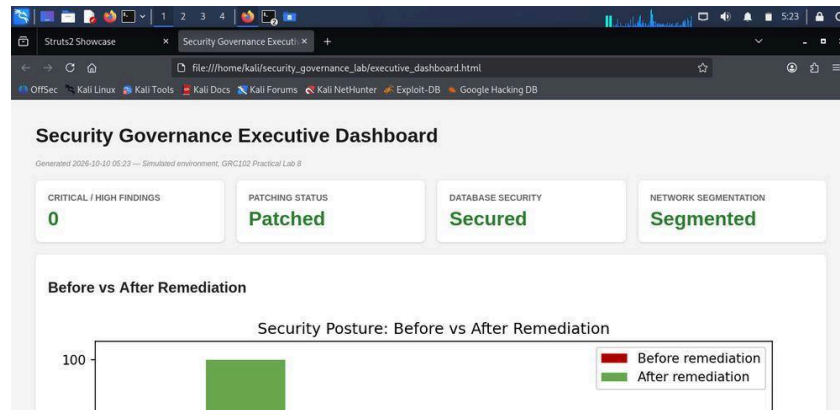


Figure 4.14: Fixing a shell syntax error while writing the dashboard user guide

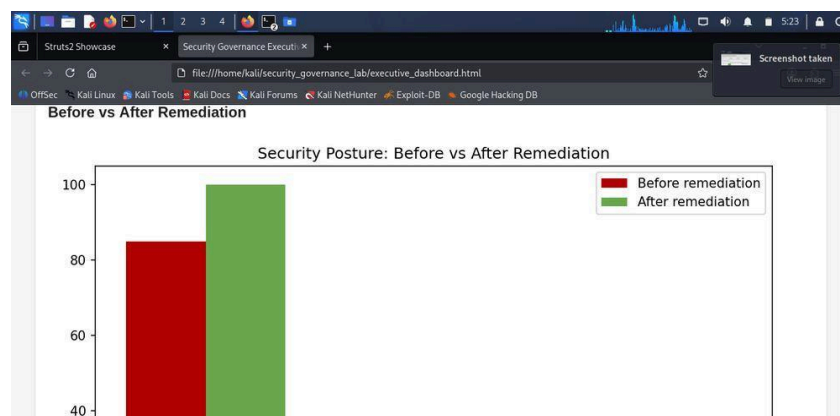


Figure 4.15: Fixing a shell syntax error while writing the dashboard user guide

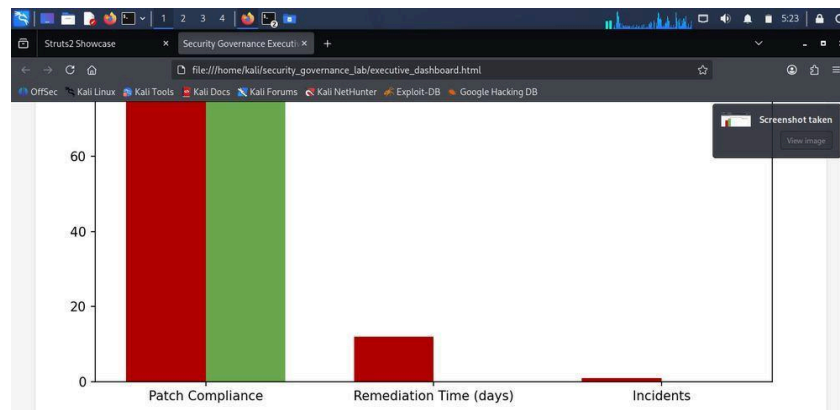


Figure 4.16: Fixing a shell syntax error while writing the dashboard user guide

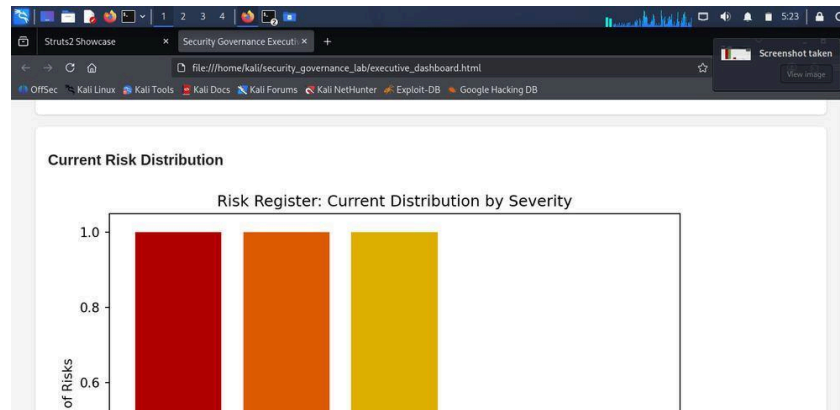


Figure 4.17: Fixing a shell syntax error while writing the dashboard user guide



Figure 4.18: Generating and opening the executive HTML dashboard

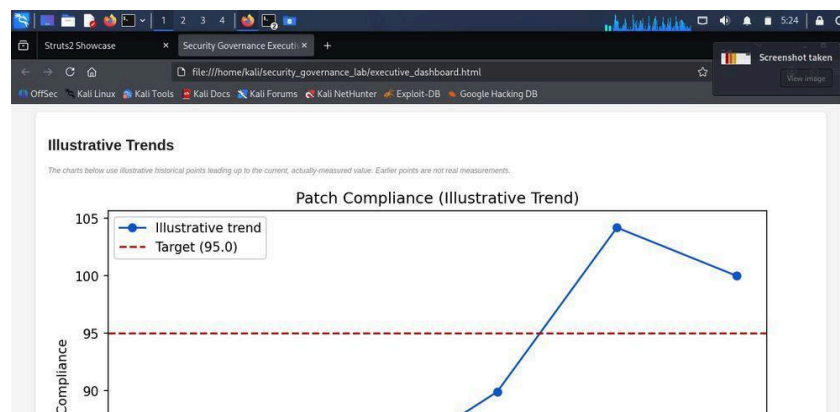


Figure 4.19: Generating and opening the executive HTML dashboard

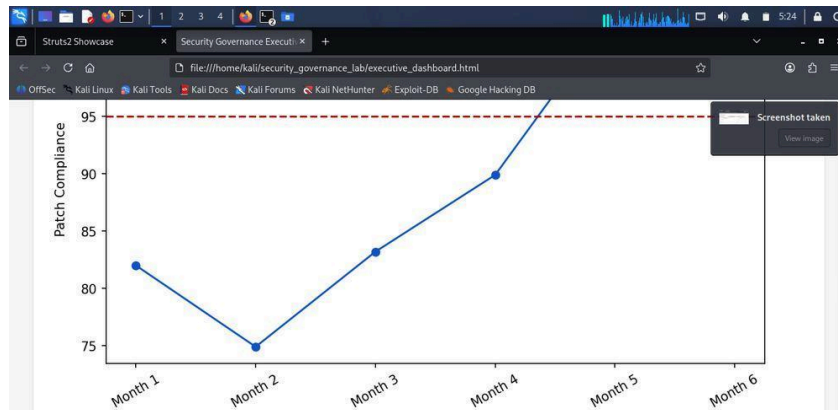


Figure 4.20: Generating and opening the executive HTML dashboard

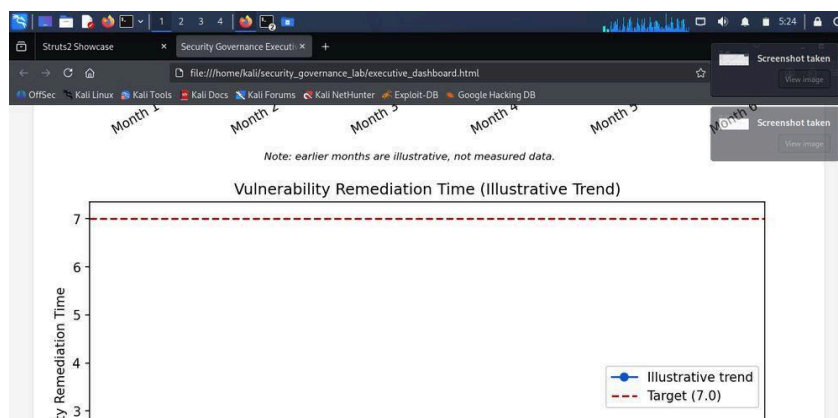


Figure 4.21: Generating and opening the executive HTML dashboard

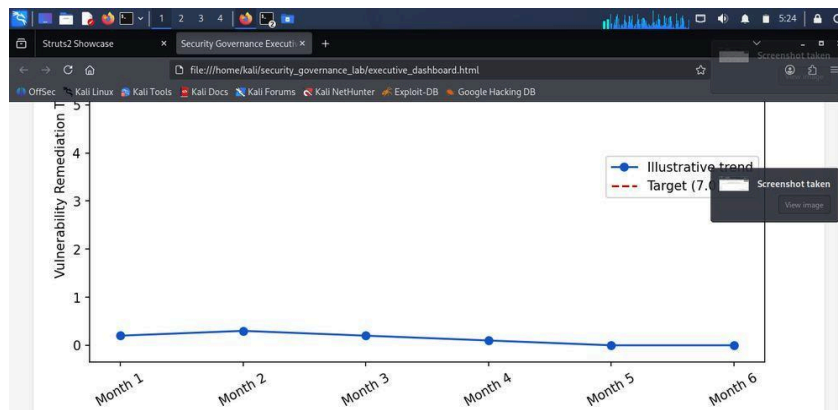


Figure 4.22: Generating and opening the executive HTML dashboard

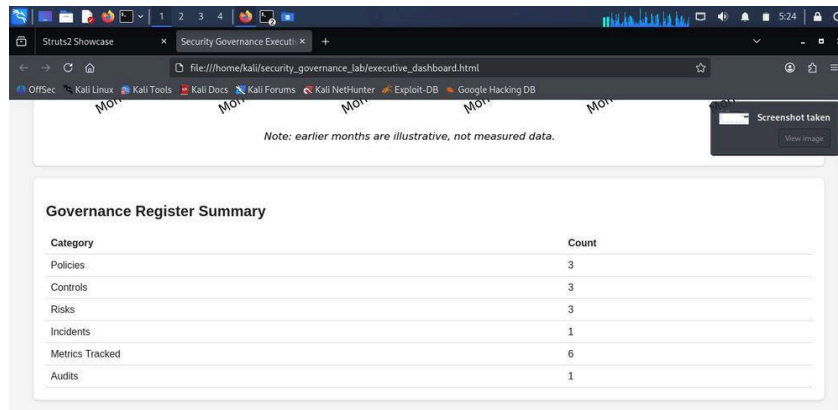


Figure 4.23: Generating and opening the executive HTML dashboard

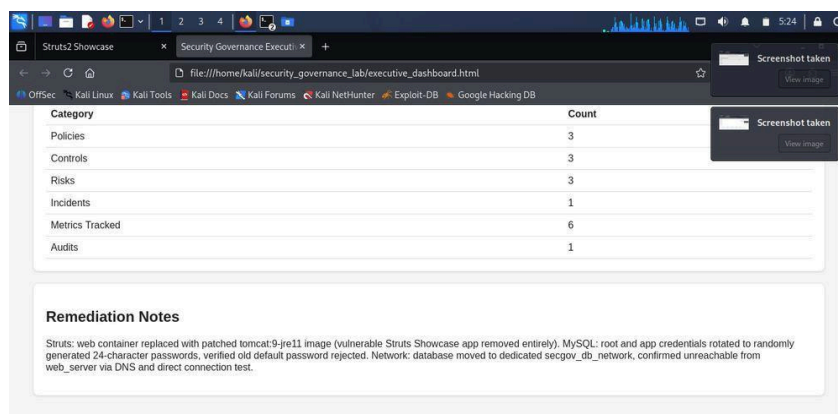


Figure 4.24: Generating and opening the executive HTML dashboard

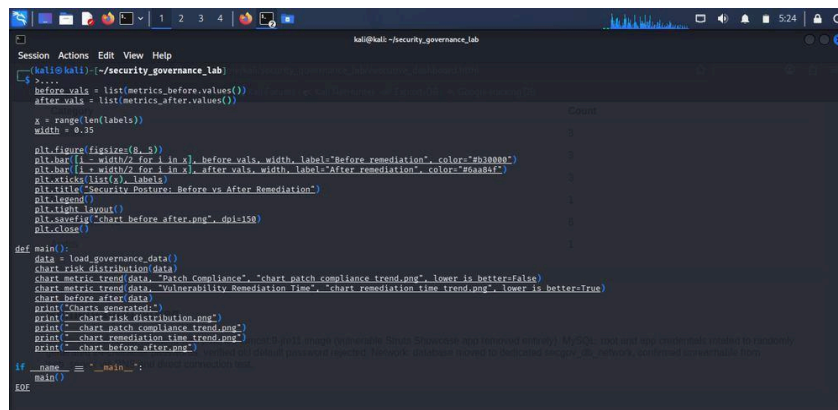


Figure 4.25: Generating and opening the executive HTML dashboard

[illegible]

Figure 4.26: *Writing the metrics framework document*

```
kali@kali:~/security_governance_lab

Session Actions Edit View Help

<tr><th>Category</th><th>Count</th></tr>
<tr><td>Policies</td><td>len(governance['policies'])</td></tr>
<tr><td>Controls</td><td>len(governance['controls'])</td></tr>
<tr><td>Risks</td><td>len(governance['risks'])</td></tr>
<tr><td>Incidents</td><td>len(governance['incidents'])</td></tr>
<tr><td>Metrics</td><td>len(governance['metrics'])</td></tr>
<tr><td>Audits</td><td>len(governance['audits'])</td></tr>
</table>
</div>

<div class="section">
<h2>Remediation Notes</h2>
<div patch-status="notes">No remediation notes recorded.</div>
</div>

</body>
</html>
<!--
-->

with open("executive_dashboard.html", "w") as f:
    f.write(html)

print("Dashboard generated: executive_dashboard.html")

if __name__ == "__main__":
    main()
EOF

chmod +x generate_dashboard.py
python3 generate_dashboard.py
kali@kali:~/security_governance_lab$ python3 -m ast.parse(open('generate_dashboard.py').read())
print('syntax ok')
generate_dashboard.py
firefox executive_dashboard.html
```

Figure 4.27: *Writing the metrics framework document*

[illegible]

Figure 4.28: *Writing the metrics framework document*

```
Session Actions Edit View Help
[1] * done      firefox executive.dashboard.html
kali@kali:~/security_governance_lab$ cat metrics_framework.md
# Security Metrics Framework

## Purpose

This framework defines how security posture is measured, tracked, and reported within the simulated governance program, and explains how each metric drives action rather than sitting unused.

## Metrics Tracked

| Metric | Definition | Target | Direction |
|---|---|---|---|
| Patch Compliance | Percentage of systems with all critical patches applied | 95% | Higher is better |
| Vulnerability Remediation Time | Average days to remediate a critical vulnerability | 7 days | Lower is better |
| Security Incidents | Number of security incidents per reporting period | 0 | Lower is better |

## How Status Is Determined

Each metric is scored Green or Red by comparing its actual value against its target, accounting for whether the metric is "higher is better" (like Patch Compliance) or "lower is better" (like Remediation Time and Incidents). This distinction matters: an earlier version of the tracking tool used in this lab treated every metric as "higher is better," which meant a rising incident count was incorrectly shown as Green. That bug was identified and corrected before the post-remediation metrics were recorded, so the directionality shown in this report is accurate.

## Reporting Cadence

In a production environment, these metrics would be reviewed monthly by the security team and quarterly by governance leadership, with Red status on any metric triggering a documented remediation plan and an update at the following review. In this simulation, metrics are captured at two points: before remediation (reflecting the Equifax-style baseline) and after remediation (reflecting the controls implemented in Part 3).

## Ownership

Each metric has a nominal owner recorded in the governance tracker: patch compliance and remediation time are owned by the Security Engineer role, and the incident count by the Security Director role, consistent with the governance structure recorded in the risk and control register.

## Limitation

The trend charts accompanying this framework use illustrative historical data points to visualize what a realistic trend toward the current measured value might look like. Only the final data point in each trend, and the full before/after comparison chart, reflect actual measurements taken during this exercise.

EOF
```

Figure 4.29: Writing the dashboard user guide

```
Session Actions Edit View Help
kali@kali:~/security_governance_lab$ cat metrics_framework.md
# Security Metrics Framework

## Purpose

This framework defines how security posture is measured, tracked, and reported within the simulated governance program, and explains how each metric drives action rather than sitting unused.

## Metrics Tracked

| Metric | Definition | Target | Direction |
|---|---|---|---|
| Patch Compliance | Percentage of systems with all critical patches applied | 95% | Higher is better |
| Vulnerability Remediation Time | Average days to remediate a critical vulnerability | 7 days | Lower is better |
| Security Incidents | Number of security incidents per reporting period | 0 | Lower is better |

## How Status Is Determined

Each metric is scored Green or Red by comparing its actual value against its target, accounting for whether the metric is "higher is better" (like Patch Compliance) or "lower is better" (like Remediation Time and Incidents). This distinction matters: an earlier version of the tracking tool used in this lab treated every metric as "higher is better," which meant a rising incident count was incorrectly shown as Green. That bug was identified and corrected before the post-remediation metrics were recorded, so the directionality shown in this report is accurate.

## Reporting Cadence

In a production environment, these metrics would be reviewed monthly by the security team and quarterly by governance leadership, with Red status on any metric triggering a documented remediation plan and an update at the following review. In this simulation, metrics are captured at two points: before remediation (reflecting the Equifax-style baseline) and after remediation (reflecting the controls implemented in Part 3).

## Ownership

Each metric has a nominal owner recorded in the governance tracker: patch compliance and remediation time are owned by the Security Engineer role, and the incident count by the Security Director role, consistent with the governance structure recorded in the risk and control register.

## Limitation

The trend charts accompanying this framework use illustrative historical data points to visualize what a realistic trend toward the current measured value might look like. Only the final data point in each trend, and the full before/after comparison chart, reflect actual measurements taken during this exercise.

EOF
```

Figure 4.30: Writing the dashboard user guide

```
Session Actions Edit View Help
kali@kali:~/security_governance_lab$ cat metrics_framework.md
# Security Metrics Framework

## Purpose

This framework defines how security posture is measured, tracked, and reported within the simulated governance program, and explains how each metric drives action rather than sitting unused.

## Metrics Tracked

| Metric | Definition | Target | Direction |
|---|---|---|---|
| Patch Compliance | Percentage of systems with all critical patches applied | 95% | Higher is better |
| Vulnerability Remediation Time | Average days to remediate a critical vulnerability | 7 days | Lower is better |
| Security Incidents | Number of security incidents per reporting period | 0 | Lower is better |

## How Status Is Determined

Each metric is scored Green or Red by comparing its actual value against its target, accounting for whether the metric is "higher is better" (like Patch Compliance) or "lower is better" (like Remediation Time and Incidents). This distinction matters: an earlier version of the tracking tool used in this lab treated every metric as "higher is better," which meant a rising incident count was incorrectly shown as Green. That bug was identified and corrected before the post-remediation metrics were recorded, so the directionality shown in this report is accurate.

## Reporting Cadence

In a production environment, these metrics would be reviewed monthly by the security team and quarterly by governance leadership, with Red status on any metric triggering a documented remediation plan and an update at the following review. In this simulation, metrics are captured at two points: before remediation (reflecting the Equifax-style baseline) and after remediation (reflecting the controls implemented in Part 3).

## Ownership

Each metric has a nominal owner recorded in the governance tracker: patch compliance and remediation time are owned by the Security Engineer role, and the incident count by the Security Director role, consistent with the governance structure recorded in the risk and control register.

## Limitation

The trend charts accompanying this framework use illustrative historical data points to visualize what a realistic trend toward the current measured value might look like. Only the final data point in each trend, and the full before/after comparison chart, reflect actual measurements taken during this exercise.

kali@kali:~/security_governance_lab$
```

Figure 4.31: Writing the dashboard user guide

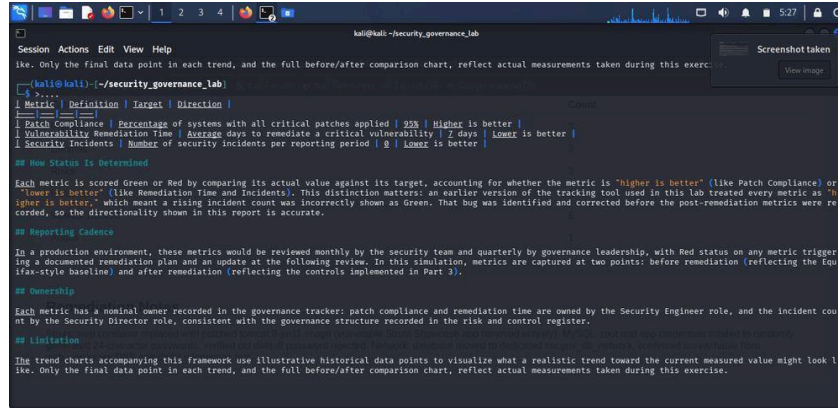


Figure 4.32: Writing the dashboard user guide

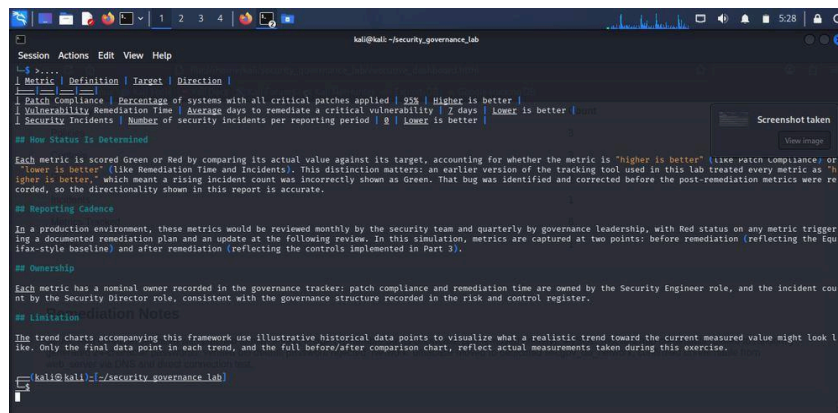


Figure 4.33: Writing the board report

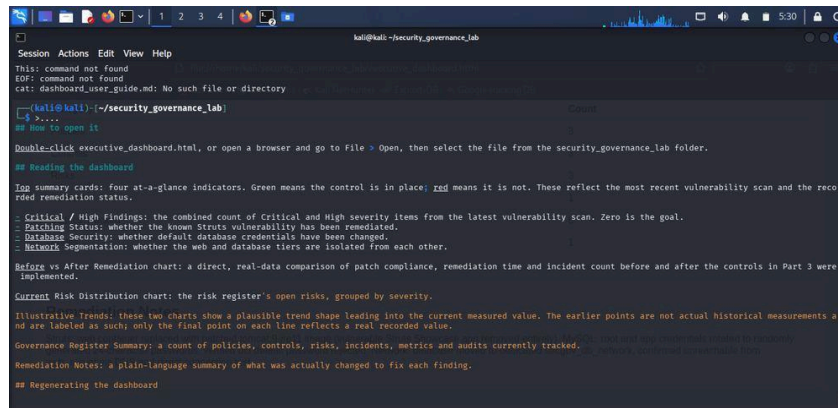


Figure 4.34: Reviewing the board report content

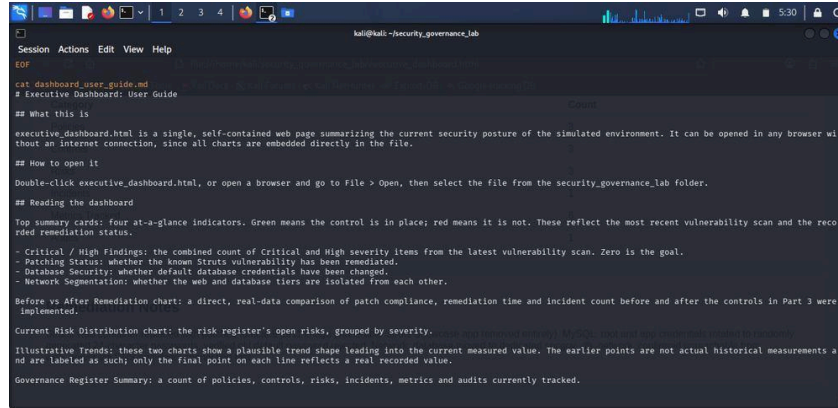


Figure 4.35: Reviewing the board report content

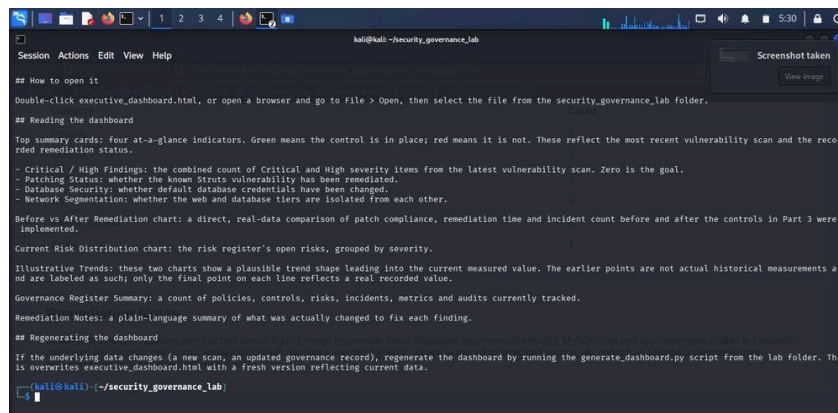


Figure 4.36: Reviewing the board report content

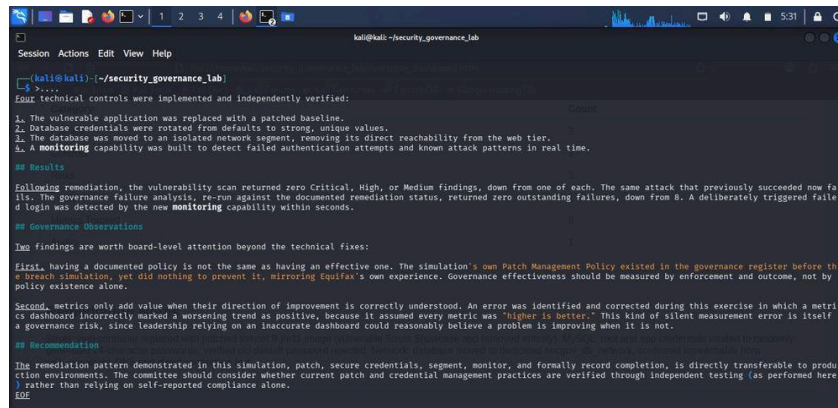


Figure 4.37: Finalizing the board report

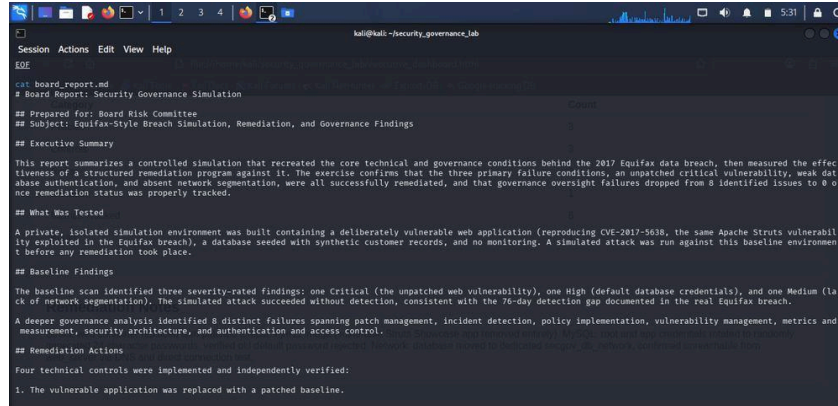


Figure 4.38: Finalizing the board report

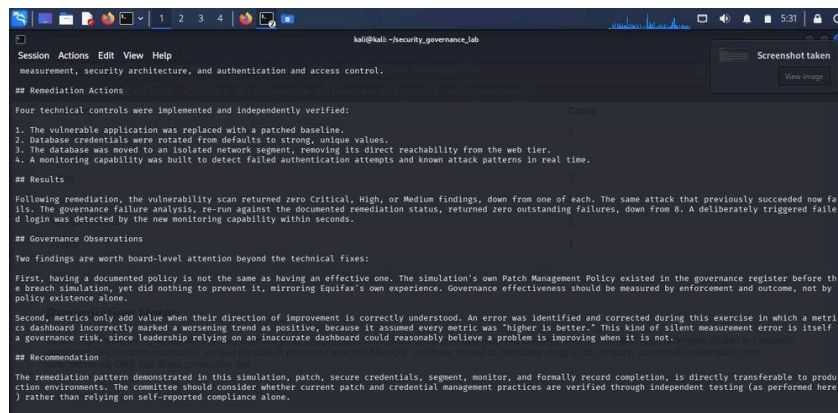


Figure 4.39: Finalizing the board report

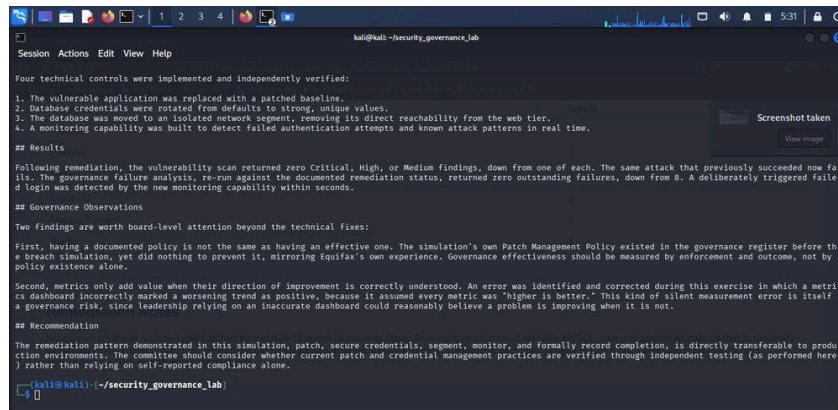


Figure 4.40: Finalizing the board report

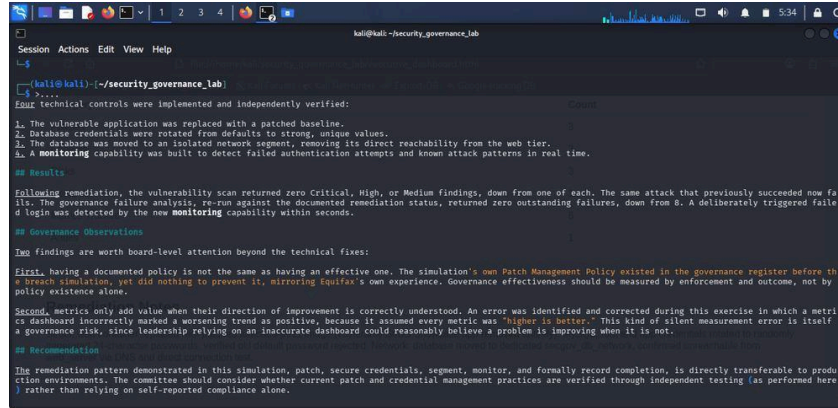


Figure 4.41: Viewing the completed executive dashboard in Firefox

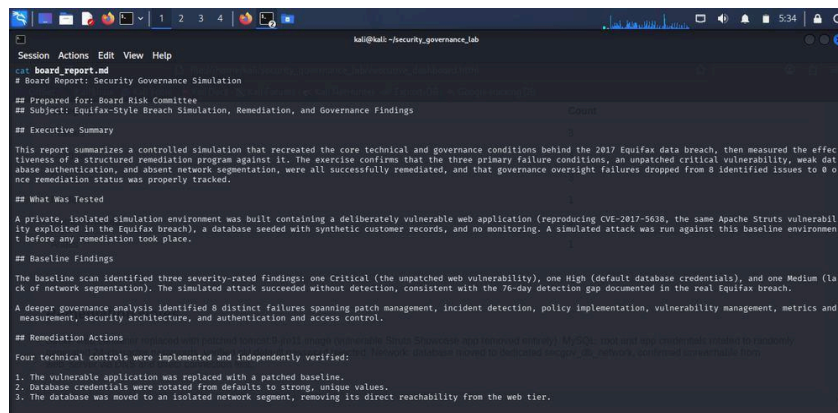


Figure 4.42: Viewing the completed executive dashboard in Firefox

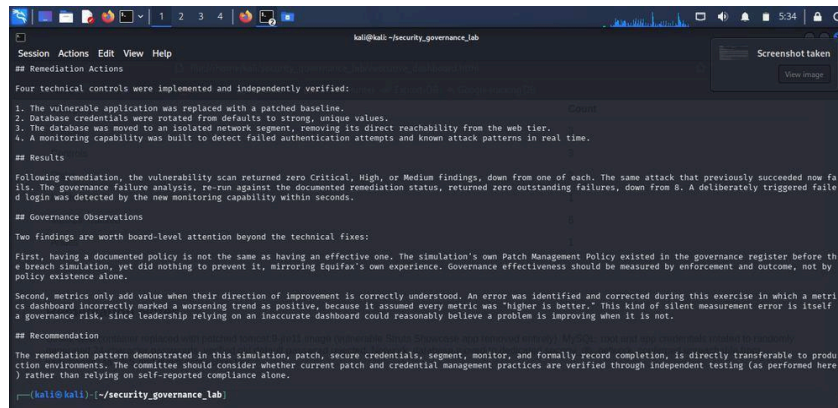


Figure 4.43: Viewing the completed executive dashboard in Firefox

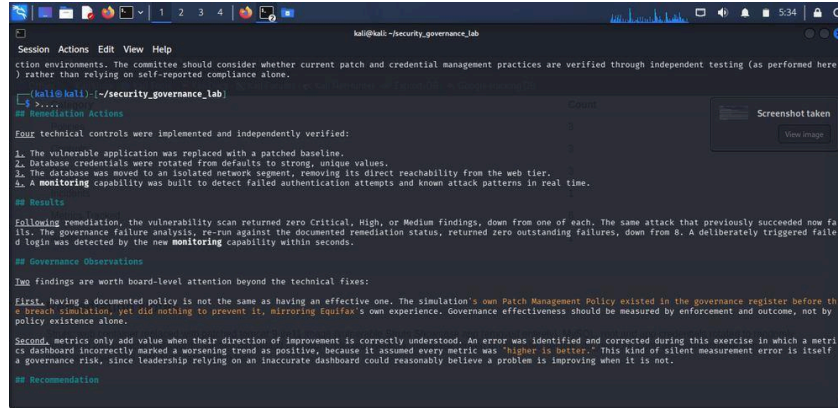


Figure 4.44: Viewing the completed executive dashboard in Firefox

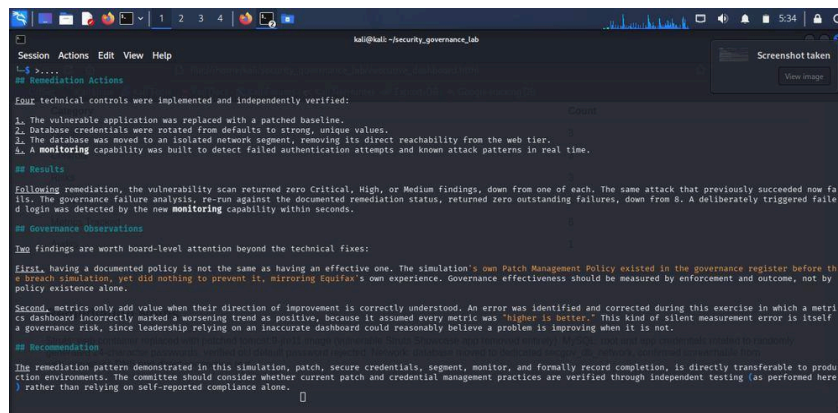


Figure 4.45: Viewing the completed executive dashboard in Firefox